

Each topic follow this format:

- **Question**
 - **Answer**
 - **Key Points**
 - **Example**
 - **Interview Follow-up**
-

Chapter 1 — OS Fundamentals (Q1–Q5)

1. What is an Operating System, and what are its primary responsibilities?

Answer

An **Operating System (OS)** is system software that acts as an intermediary between computer hardware and application programs. It manages hardware resources, provides common services to applications, and creates an environment where multiple programs can run efficiently and securely.

Without an operating system, every application would need to communicate directly with hardware devices such as the CPU, memory, storage, and input/output devices. The OS abstracts this complexity by providing standardized interfaces.

Modern operating systems also ensure that multiple users and applications can safely share system resources without interfering with each other.

Primary Responsibilities

- Managing CPU execution
 - Managing main memory (RAM)
 - Managing files and storage
 - Handling input/output devices
 - Scheduling processes
 - Providing security and user authentication
 - Allocating hardware resources
 - Handling errors and recovery
-

Example

When you open Chrome:

1. The OS loads Chrome into memory.
2. CPU time is allocated.

3. Required files are accessed from storage.
4. Keyboard and mouse inputs are handled.
5. Network communication is managed.
6. Memory is released after Chrome exits.

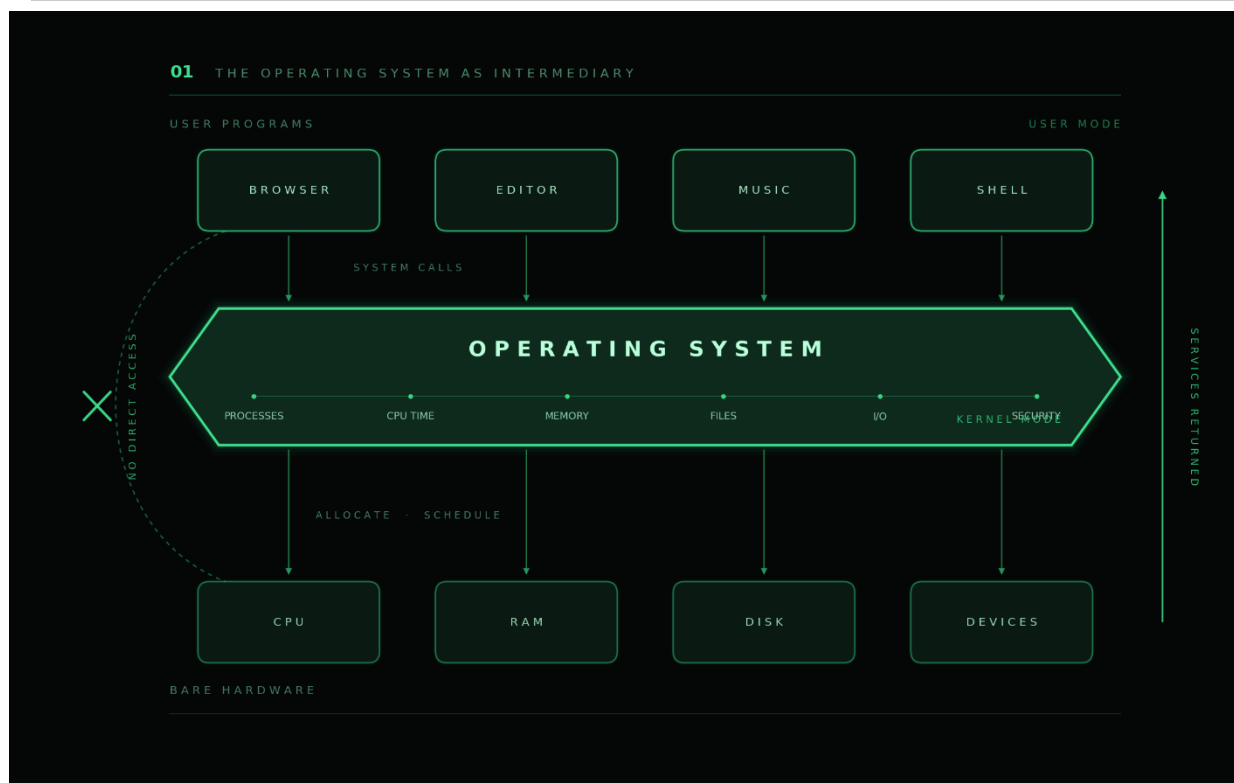
Key Points

- Acts as the interface between hardware and software.
 - Controls all hardware resources.
 - Enables multitasking.
 - Ensures security and resource isolation.
-

Interview Follow-up

Q: Can a computer run without an operating system?

A: Yes, but only for special-purpose software that directly controls hardware (embedded firmware or bootloaders). General-purpose applications require an operating system.



2. What are the different types of operating systems? Give examples of each.

Answer

Operating systems are categorized based on how they manage resources and serve users.

Batch Operating System

Jobs are collected into batches and executed sequentially without user interaction.

Characteristics

- No direct user interaction
- High throughput
- Long waiting time

Example

Early IBM mainframe systems

Time-Sharing Operating System

Multiple users share CPU time through rapid context switching.

Characteristics

- Interactive
- Fast response
- Supports multiple users

Examples

- Linux
 - UNIX
-

Multiprogramming Operating System

Several programs remain in memory simultaneously.

Whenever one process waits for I/O, another process uses the CPU.

This improves CPU utilization.

Multitasking Operating System

Allows a single user to execute multiple applications simultaneously.

Example

Running:

- Browser
- VS Code
- Spotify
- Terminal

at the same time.

Multiprocessing Operating System

Uses multiple CPUs or multiple cores.

Advantages:

- Better performance
 - Higher reliability
 - Parallel execution
-

Distributed Operating System

Several computers work together and appear as one logical system.

Advantages:

- Resource sharing
 - Fault tolerance
 - Scalability
-

Real-Time Operating System (RTOS)

Provides deterministic responses within strict deadlines.

Types:

Hard RTOS

Missing a deadline is unacceptable.

Example:

Aircraft control systems

Soft RTOS

Occasional delays are acceptable.

Example:

Video streaming

Key Points

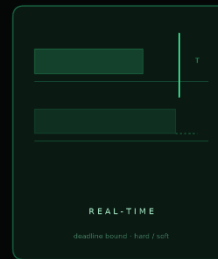
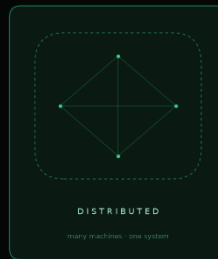
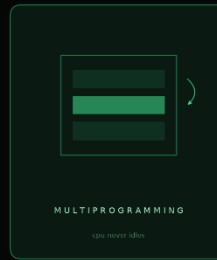
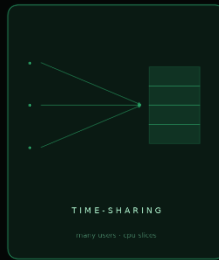
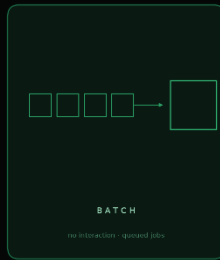
- Batch → No interaction
 - Time-sharing → Multiple users
 - Multiprogramming → Better CPU utilization
 - Multitasking → Multiple applications
 - Multiprocessing → Multiple CPUs
 - Distributed → Multiple computers
 - RTOS → Time-critical systems
-

Interview Follow-up

Q: Which operating system type is Windows?

A: Windows is primarily a multitasking, multiprocessing, time-sharing operating system.

CLASSIFIED BY HOW RESOURCES ARE SHARED



3. What is the difference between the kernel and an operating system?

Answer

The **Operating System** is the complete software package that manages a computer.

The **Kernel** is the core component of the operating system responsible for direct interaction with hardware.

The kernel always remains loaded in memory, while other OS components may load only when required.

Responsibilities of the Kernel

- CPU scheduling
- Memory management
- Device management
- Interrupt handling
- System call processing
- Process management

Applications cannot directly access hardware. Instead, they request services through the kernel.

Comparison

Operating System	Kernel
Complete system software	Core component
Includes utilities and user interface	Handles hardware interactions
Provides services to users	Provides services to applications
Larger in scope	Smaller but critical

Example

When a program wants to read a file:

Application

↓

System Call

↓

Kernel

↓

Disk Controller



Disk



Kernel



Application

The application never communicates directly with the hardware.

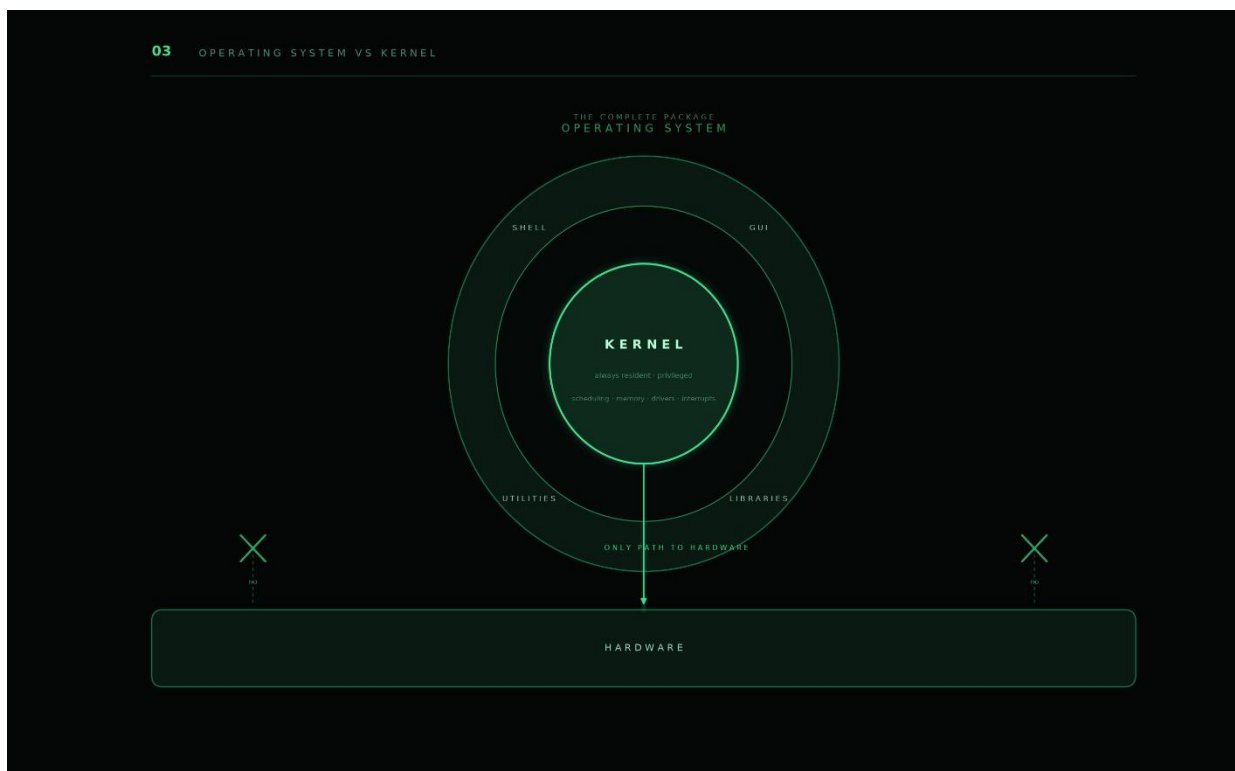
Key Points

- Kernel is part of the OS.
 - Every OS has a kernel.
 - Kernel operates in privileged mode.
-

Interview Follow-up

Q: Can the kernel run without an operating system?

A: No. The kernel itself is the core of the operating system.



4. What is a system call? Explain how user programs interact with the kernel.

Answer

A **system call** is the mechanism through which a user program requests services from the operating system kernel.

Applications operate in **user mode**, where direct access to hardware is restricted. Whenever privileged operations are needed, the application invokes a system call.

Examples include:

- Creating a process
- Reading a file
- Writing to a file
- Allocating memory
- Opening a network socket

How It Works

1. Application requests a service.
 2. A system call is issued.
 3. CPU switches from user mode to kernel mode.
 4. Kernel performs the requested operation.
 5. Control returns to user mode.
-

Example

When executing:

```
fstream file("notes.txt");
```

Internally, the OS performs:

- Open file
 - Locate inode or file metadata
 - Check permissions
 - Read data from disk
 - Return a file descriptor
-

Common Categories

Process Control

- Create process
 - Terminate process
 - Wait for child
-

File Management

- Open
 - Read
 - Write
 - Close
-

Device Management

- Read from keyboard
- Print to screen

Information Maintenance

- Get system time
 - Get process ID
-

Communication

- Pipes
 - Shared memory
 - Sockets
-

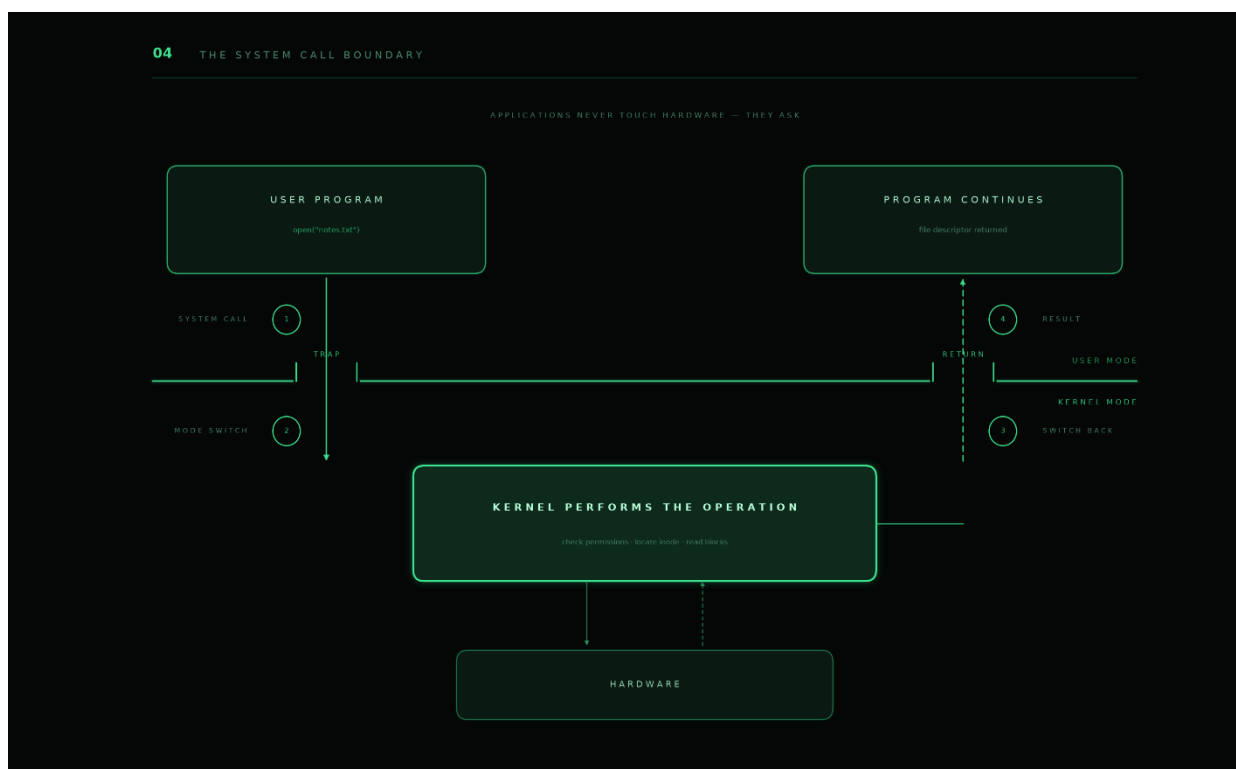
Key Points

- System calls are the interface between applications and the kernel.
 - They provide controlled access to hardware.
 - They prevent applications from directly manipulating hardware.
-

Interview Follow-up

Q: Why can't applications access hardware directly?

A: Direct hardware access would compromise system stability and security. Restricting hardware access to the kernel ensures controlled resource management and isolation between applications.



5. What happens internally from the moment you press the power button until the operating system becomes usable?

Answer

The boot process is the sequence of steps that prepares a computer for use.

Step 1 — Power On

The CPU resets and begins executing firmware stored in ROM or flash memory.

Step 2 — BIOS or UEFI

The firmware performs a **Power-On Self-Test (POST)** to verify that essential hardware components, such as memory, CPU, and storage devices, are functioning correctly.

It then identifies a bootable storage device.

Step 3 — Bootloader

The bootloader (such as GRUB on Linux or Windows Boot Manager) is loaded into memory.

Its job is to locate and load the operating system kernel.

Step 4 — Kernel Initialization

The kernel initializes critical system components:

- Memory manager
 - Process scheduler
 - Device drivers
 - Interrupt handlers
 - File systems
-

Step 5 — System Initialization

The kernel starts the first user-space process.

On Linux, this is typically systemd or init.

This process starts:

- Login services
 - Networking
 - Background services
 - Device management
-

Step 6 — User Login

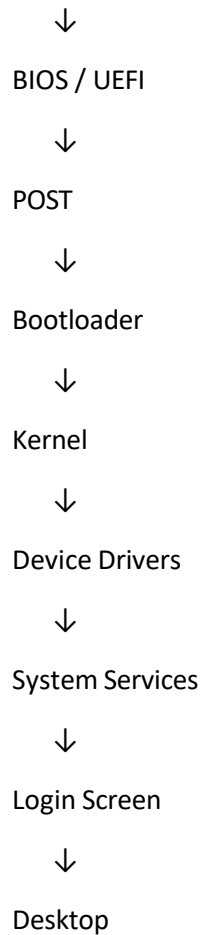
The login screen appears.

After successful authentication:

- User profile is loaded
 - Desktop environment starts
 - Applications become available
-

Boot Flow

Power Button



Key Points

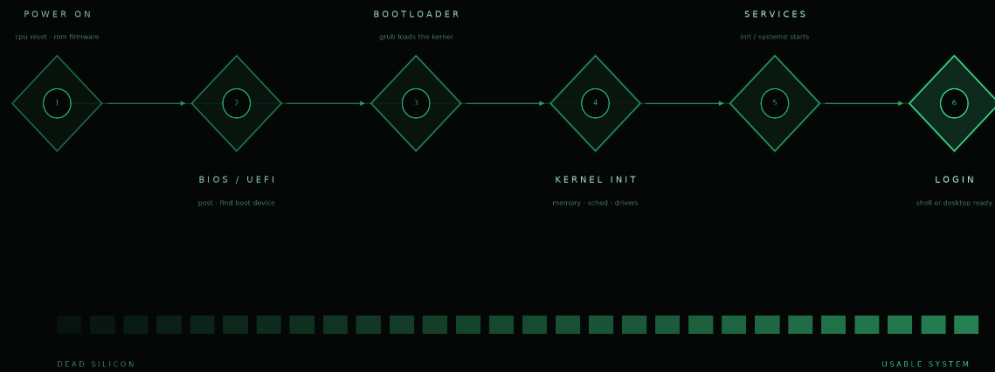
- BIOS/UEFI initializes hardware.
 - The bootloader loads the kernel.
 - The kernel initializes core system components.
 - User-space services start after kernel initialization.
 - The system becomes usable once login and essential services are ready.
-

Interview Follow-up

Q: What is the difference between BIOS and UEFI?

A: BIOS is the traditional firmware interface with limited features, while UEFI is its modern replacement. UEFI supports larger disks, faster boot times, graphical interfaces, secure boot, and more flexible boot management.

FIRMWARE HANDS OFF TO KERNEL, KERNEL HANDS OFF TO USER SPACE



Chapter 2 — Processes & Threads (Q6–Q14)

Processes and threads are among the most frequently asked operating system topics in interviews. A solid understanding of how programs execute, how the OS manages them, and how concurrency works is essential.

6. What is a process? How is it represented by the operating system?

Answer

A **process** is an instance of a program that is currently being executed. While a program is a passive file stored on disk, a process is an active entity with its own execution state, memory, and system resources.

Every time you launch an application, the operating system creates a new process for it.

For example, if you open three instances of a text editor, the operating system creates three separate processes, even though they all execute the same program.

Each process has its own:

- Process ID (PID)
- Program Counter (PC)
- CPU registers
- Stack
- Heap
- Code (Text) segment
- Data segment
- Open files
- Security information

Process Memory Layout

```
+-----+
|  Stack  |
| (Function Calls) |
+-----+
|  Heap   |
| (Dynamic Memory) |
+-----+
| Initialized Data |
+-----+
| Uninitialized Data |
|  (BSS)   |
+-----+
```

| Code (Text Segment) |

+-----+

Example

Suppose you run:

Google Chrome

The operating system:

- Creates a process.
- Allocates memory.
- Assigns a unique PID.
- Loads the executable into RAM.
- Schedules CPU time.
- Starts execution.

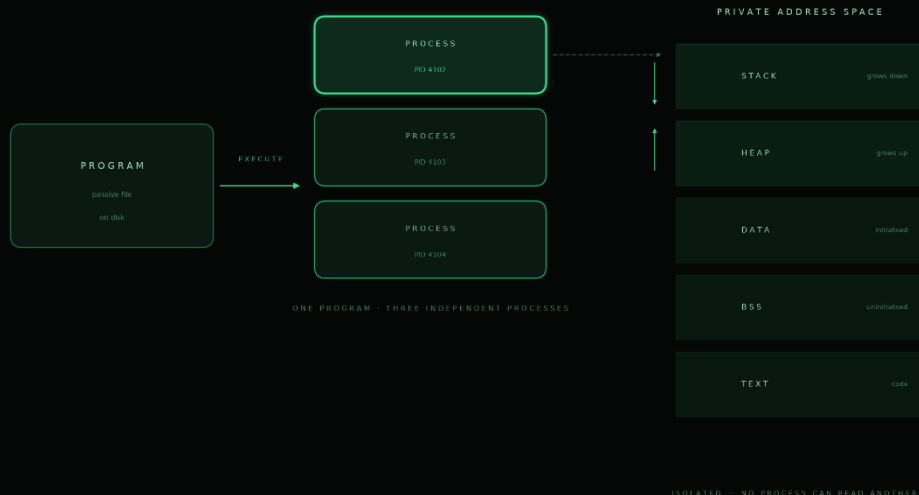
Key Points

- A process is an executing program.
- Multiple processes can run the same program independently.
- Each process has its own memory and execution context.
- Processes are isolated from one another.

Interview Follow-up

Q: Can two processes access each other's memory?

A: Not directly. Each process has its own protected address space. Communication requires mechanisms such as shared memory, pipes, sockets, or message queues.



7. What are the different states of a process? Explain every state transition.

Answer

During its lifetime, a process moves through several execution states depending on CPU availability and system events.

Process States

1. New

The process has been created but is not yet ready for execution.

2. Ready

The process has everything it needs except the CPU.

It waits in the **Ready Queue** until the scheduler selects it.

3. Running

The CPU is executing the process instructions.

Only one process per CPU core can be in the Running state at a time.

4. Waiting (Blocked)

The process cannot continue because it is waiting for an external event.

Examples:

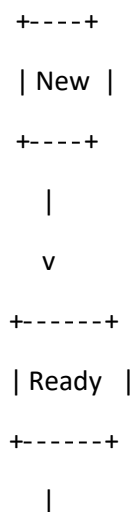
- Disk read
 - Network response
 - User input
-

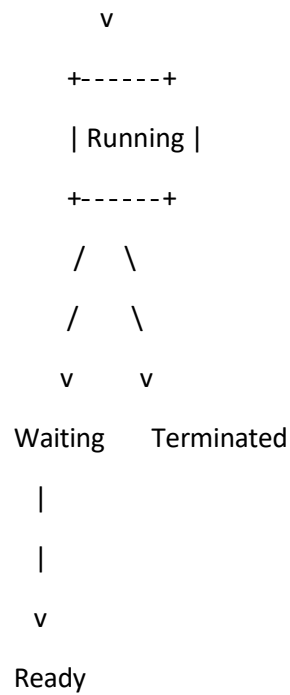
5. Terminated

The process has completed execution or has been killed.

The operating system releases all allocated resources.

State Diagram





Example

Imagine downloading a file.

- Process starts → New
- Waits for CPU → Ready
- CPU executes → Running
- Waiting for internet packets → Waiting
- Packets received → Ready
- CPU resumes → Running
- Download finishes → Terminated

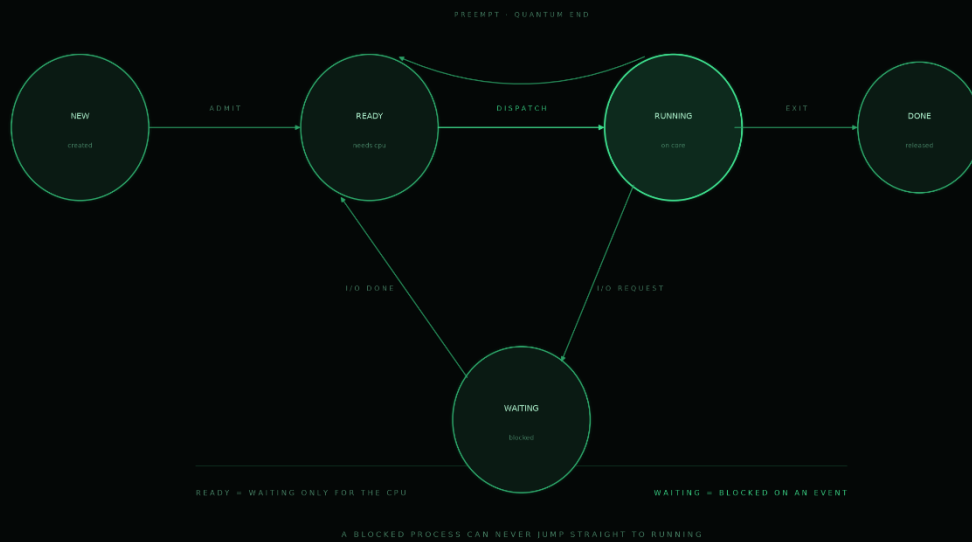
Key Points

- Ready ≠ Waiting.
- Ready means waiting only for CPU.
- Waiting means blocked by an external event.

Interview Follow-up

Q: Can a process move directly from Waiting to Running?

A: No. It first enters the Ready queue and must be selected by the CPU scheduler.



8. What is a Process Control Block (PCB)? What information does it store?

Answer

A **Process Control Block (PCB)** is a data structure maintained by the operating system that stores all information needed to manage a process.

Whenever the CPU switches between processes, the OS saves the current process's state in its PCB and restores another process's state from its PCB.

Without the PCB, multitasking would not be possible.

Information Stored in PCB

- Process ID (PID)
- Current process state
- Program Counter
- CPU registers
- CPU scheduling information
- Memory management information
- Open file descriptors
- I/O status
- Accounting information

Visual Representation

PCB

PID

Process State

Program Counter

CPU Registers

Memory Information

Scheduling Information

Open Files

I/O Status

Example

Suppose Process A is interrupted.

The OS stores:

- Register values
- Current instruction
- Stack pointer

inside its PCB.

Later, when Process A resumes, execution continues exactly where it stopped.

Key Points

- Every process has one PCB.
- PCB enables context switching.
- PCB stores both execution state and resource information.

Interview Follow-up

Q: Where is the PCB stored?

A: It resides in kernel memory because only the operating system should modify process management data.



9. What is the difference between a process and a thread?

Answer

A **process** is an independent execution unit with its own memory space.

A **thread** is the smallest unit of execution within a process.

Multiple threads inside a process share resources but execute independently.

Comparison

Process	Thread
Own memory	Shared memory
Heavyweight	Lightweight
Slower creation	Faster creation

Process	Thread
Expensive context switch	Cheaper context switch
Better isolation	Easier communication
Crash usually isolated	Thread crash may affect the process

Example

Google Chrome:

One process may contain multiple threads:

- UI Thread
- Rendering Thread
- Network Thread
- JavaScript Engine Thread

These threads work simultaneously while sharing the same memory.

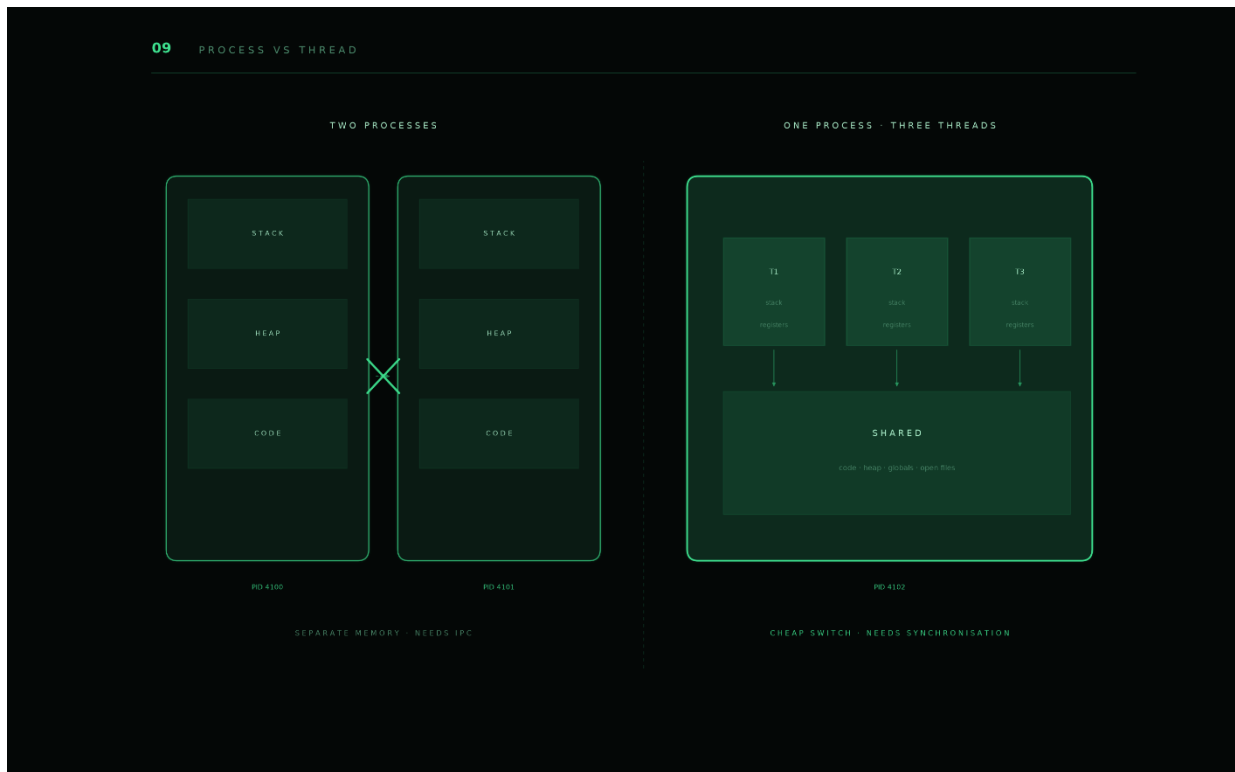
Key Points

- Threads share code, heap, and global variables.
 - Threads have separate stacks and registers.
 - Thread communication is easier but requires synchronization.
-

Interview Follow-up

Q: Why are threads faster than processes?

A: Threads share the same address space, eliminating the need for expensive memory setup and reducing context-switch overhead.



10. What is the difference between user-level threads and kernel-level threads?

Answer

Threads can be managed either by a user-space library or directly by the operating system.

User-Level Threads (ULT)

Managed entirely by a thread library.

The operating system is unaware of individual threads.

Advantages

- Fast creation
- Fast switching
- Portable

Disadvantages

- Blocking system calls block all threads.
 - Cannot fully utilize multiple CPU cores in many implementations.
-

Kernel-Level Threads (KLT)

Managed directly by the operating system.

Each thread is visible to the scheduler.

Advantages

- True parallel execution
- Better scheduling
- One blocked thread does not stop others

Disadvantages

- Higher overhead
 - Slower creation
 - Slower context switching
-

Key Points

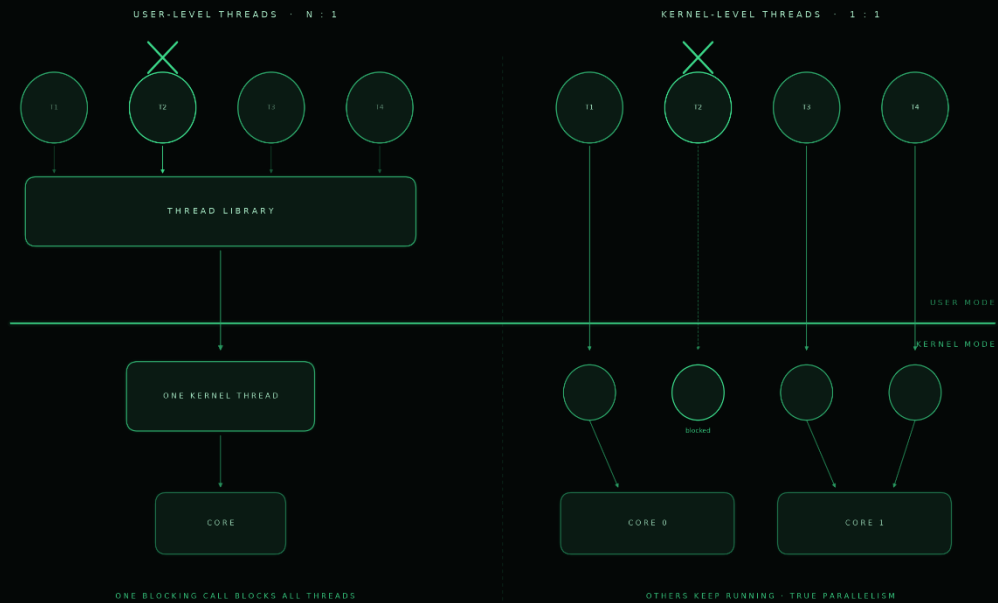
- ULT → Managed in user space.
 - KLT → Managed by the kernel.
 - Modern operating systems primarily use kernel threads.
-

Interview Follow-up

Q: Which type is used by Linux?

A: Linux uses kernel-level threads.

10 USER-LEVEL VS KERNEL-LEVEL THREADS



11. What is context switching? Why is it considered expensive?

Answer

A **context switch** occurs when the CPU stops executing one process (or thread) and starts executing another.

The operating system must save the execution state of the current process and restore the state of the next one.

Steps

1. Save registers.
 2. Save Program Counter.
 3. Update PCB.
 4. Select next process.
 5. Load registers.
 6. Restore Program Counter.
 7. Resume execution.
-

Why Is It Expensive?

During a context switch:

- No useful application work is performed.
- CPU caches may become invalid.
- TLB entries may be flushed.
- Memory mappings may change.

Frequent context switching reduces overall system performance.

Example

Suppose:

Process A

↓

Timer Interrupt

↓

Process B

The CPU must preserve A's execution before running B.

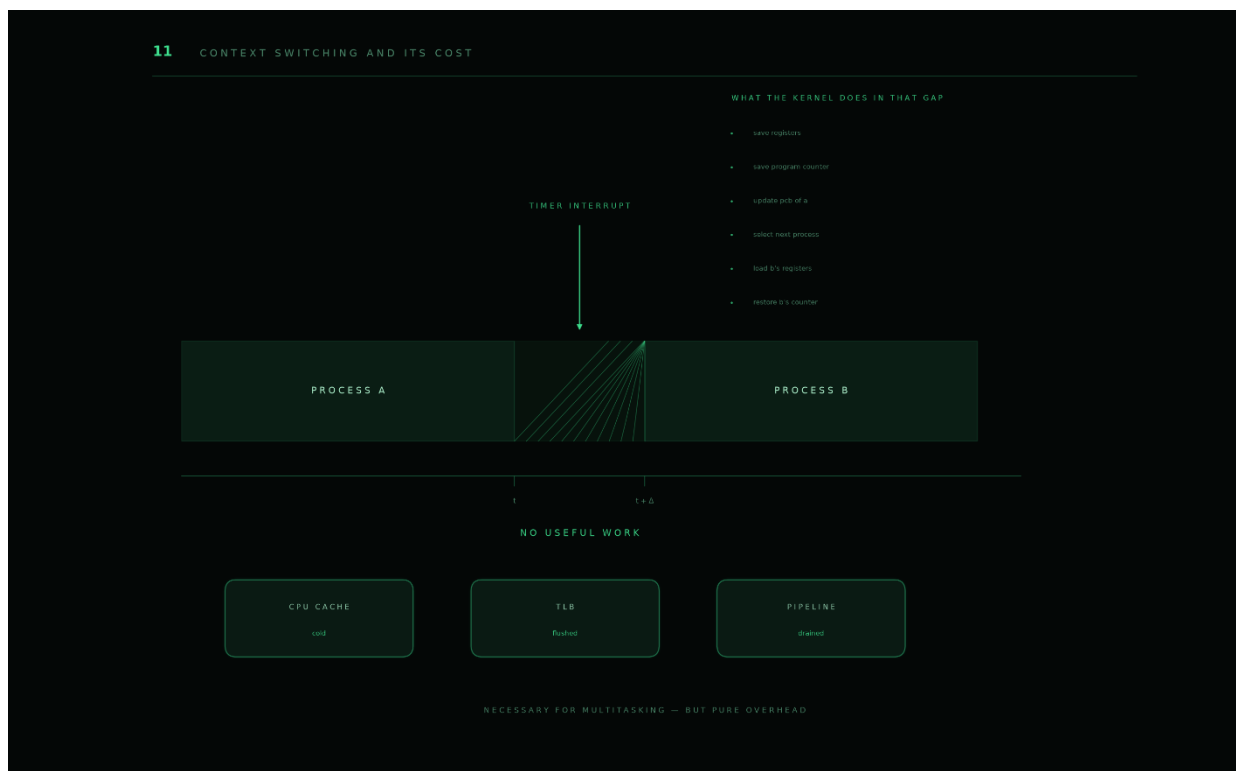
Key Points

- Necessary for multitasking.
- Does not perform productive computation.
- Lower context-switch frequency generally improves performance.

Interview Follow-up

Q: Is switching between threads cheaper than switching between processes?

A: Yes. Threads share the same address space, reducing the amount of state that must be changed.



12. What is a zombie process? How is it created and removed?

Answer

A **zombie process** is a process that has completed execution but still has an entry in the process table because its parent has not yet collected its exit status.

The process itself no longer runs or consumes CPU time, but its PID and process table entry remain.

Why Does It Exist?

When a child process terminates:

- The OS stores its exit status.
- The parent must retrieve this status using `wait()` or `waitpid()`.

- Until then, the child remains a zombie.

Example

Parent

|

+ --- Child finishes

|

Parent forgets wait()



Zombie

Removal

The parent process should call:

- wait()
- waitpid()

If the parent terminates, the init (or systemd) process adopts the child and cleans it up.

Key Points

- Zombie = Dead process with remaining metadata.
 - Consumes almost no resources except a process table entry.
 - Too many zombies can exhaust available PIDs.
-

Interview Follow-up

Q: Can a zombie process become active again?

A: No. It has already finished execution.



HARMLESS ONE AT A TIME — THOUSANDS EXHAUST THE PID SPACE

IF THE PARENT DIES FIRST, INIT ADOPTS AND REAPS IT

13. What is an orphan process?

Answer

An **orphan process** is a running process whose parent terminates before it does.

Instead of being destroyed, the orphan is adopted by the system's init process (or systemd in modern Linux systems).

The new parent becomes responsible for cleaning it up after it exits.

Example

Parent

|

+ --- Child

Parent exits

↓

Child continues

↓

Adopted by init

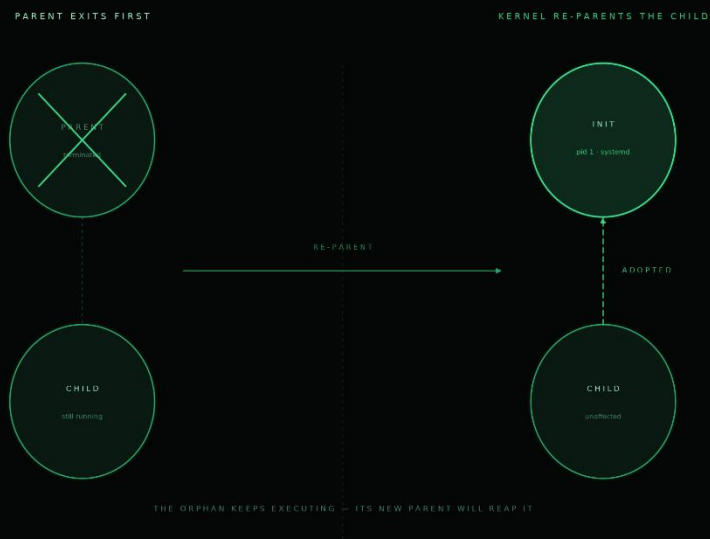
Key Points

- Orphan processes continue executing normally.
 - They are automatically adopted.
 - Adoption prevents resource leaks.
-

Interview Follow-up

Q: Is an orphan process harmful?

A: No. The operating system handles orphan adoption automatically.



14. Explain process creation and process termination in modern operating systems.

Answer

Process Creation

When a new process is created, the operating system performs several steps:

1. Assigns a new PID.
2. Creates a PCB.
3. Allocates memory.

4. Loads the executable.
5. Initializes CPU registers.
6. Places the process in the Ready queue.

On UNIX-like systems, this is commonly done using:

- `fork()` to create a child process.
 - `exec()` to replace the child's memory with a new program.
-

Process Termination

A process may terminate because:

- It finishes successfully.
- It encounters a fatal error.
- It is terminated by another process.
- The operating system shuts down.

During termination, the OS:

- Releases memory.
 - Closes open files.
 - Frees I/O resources.
 - Removes scheduling information.
 - Keeps the exit status until the parent collects it.
-

Example

Parent

|

`fork()`

|

Child

|

exec()

|

Runs New Program

|

Exit

|

Parent waits()

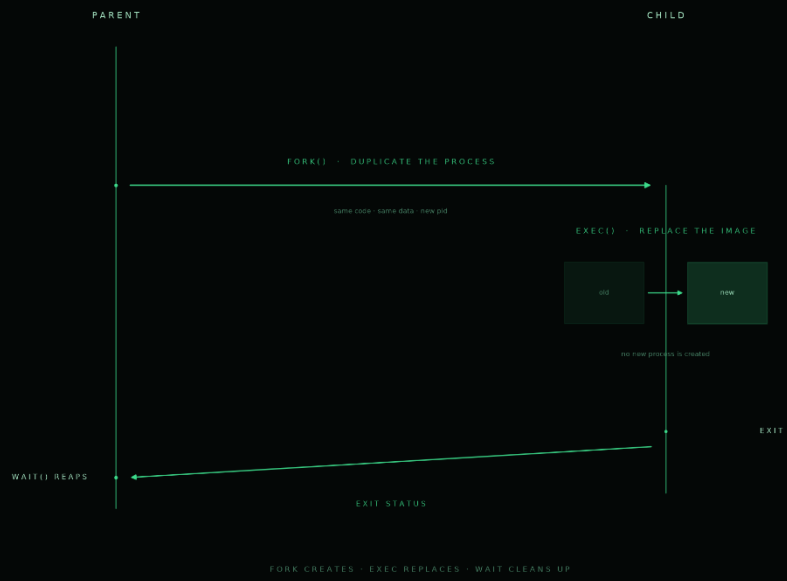
Key Points

- fork() creates a new process.
 - exec() replaces the current process image with a new program.
 - Proper termination ensures resources are released.
 - Parent processes should collect child exit status to avoid zombies.
-

Interview Follow-up

Q: What is the difference between fork() and exec()?

A: fork() creates a new process by duplicating the calling process, while exec() replaces the current process's memory with a different executable without creating a new process.



Chapter 3 — CPU Scheduling (Q15–Q22)

CPU scheduling determines **which process gets the CPU next**. Since there are usually more ready processes than available CPU cores, the operating system uses scheduling algorithms to maximize CPU utilization, improve responsiveness, and ensure fairness.

15. Why is CPU scheduling needed?

Answer

The CPU is one of the most valuable resources in a computer system. At any given moment, multiple processes may be ready to execute, but only one process per CPU core can run at a time. CPU scheduling decides **which ready process should execute next**.

Without scheduling, processes would compete uncontrollably for CPU time, leading to poor performance, unfair resource allocation, and reduced responsiveness.

The scheduler aims to keep the CPU busy while ensuring that all processes receive a fair opportunity to execute.

Objectives of CPU Scheduling

- Maximize CPU utilization
 - Increase throughput
 - Minimize waiting time
 - Minimize turnaround time
 - Improve response time
 - Ensure fairness among processes
-

Example

Suppose three programs are running:

- Chrome
- VS Code
- Spotify

All three require CPU time. The scheduler rapidly switches between them, making it appear as though they are running simultaneously.

Key Points

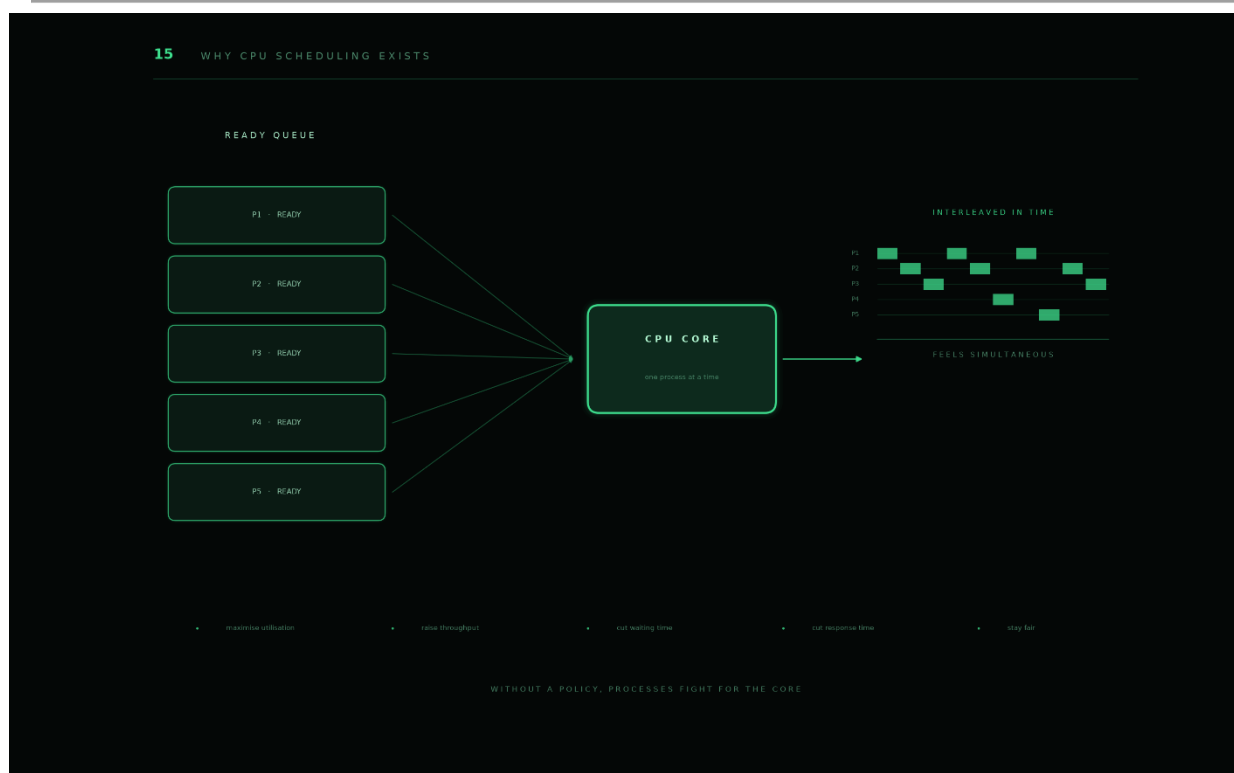
- CPU scheduling occurs whenever the CPU becomes idle or a running process is interrupted.

- It is essential for multitasking operating systems.
- Efficient scheduling improves both system performance and user experience.

Interview Follow-up

Q: Does scheduling occur on systems with multiple CPU cores?

A: Yes. Each CPU core has its own scheduling decisions, although the OS may also balance workloads across cores.



16. Compare preemptive and non-preemptive scheduling.

Answer

Scheduling algorithms are broadly classified into **preemptive** and **non-preemptive** types.

Non-Preemptive Scheduling

Once a process starts executing, it keeps the CPU until it:

- Finishes execution, or
- Enters a waiting state (such as waiting for I/O).

The operating system cannot forcibly remove it from the CPU.

Advantages

- Simple implementation
- Lower context-switch overhead

Disadvantages

- Long-running processes may delay shorter ones.
 - Poor responsiveness for interactive systems.
-

Preemptive Scheduling

The operating system can interrupt a running process and allocate the CPU to another process.

Preemption may occur due to:

- Time quantum expiration
- Arrival of a higher-priority process
- Hardware timer interrupts

Advantages

- Better responsiveness
- Improved fairness
- Suitable for interactive systems

Disadvantages

- More context switches
- Higher scheduling overhead

Comparison

Feature	Preemptive	Non-Preemptive
CPU can be taken away	Yes	No
Response time	Better	Slower
Context switches	More	Fewer
Complexity	Higher	Lower
Interactive systems	Suitable	Less suitable

Example

Imagine two processes:

- P1: 20 seconds
- P2: 2 seconds

In non-preemptive scheduling, if P1 starts first, P2 waits the full 20 seconds.

In preemptive scheduling, P2 may execute shortly after arriving.

Key Points

- Modern operating systems primarily use preemptive scheduling.
- Non-preemptive scheduling is still useful in some embedded systems.

Interview Follow-up

Q: Which scheduling type is used in Linux?

A: Linux uses preemptive scheduling.



17. Explain the FCFS scheduling algorithm with its advantages and disadvantages.

Answer

First Come First Serve (FCFS) schedules processes in the order they arrive.

The first process to enter the ready queue is the first one to execute.

It follows the **FIFO (First In, First Out)** principle.

Working

Arrival Order

P1 → P2 → P3

Execution Order

P1 → P2 → P3

Advantages

- Very simple implementation.
 - Fair in terms of arrival order.
 - Minimal scheduling overhead.
-

Disadvantages

- Poor average waiting time.
 - Long processes delay short processes.
 - Can suffer from the **Convoy Effect**.
-

Convoy Effect

A long CPU-bound process forces many short processes to wait.

Example:

P1 = 25 ms

P2 = 2 ms

P3 = 1 ms

Execution:

P1 → P2 → P3

Although P2 and P3 are short, they must wait until P1 completes.

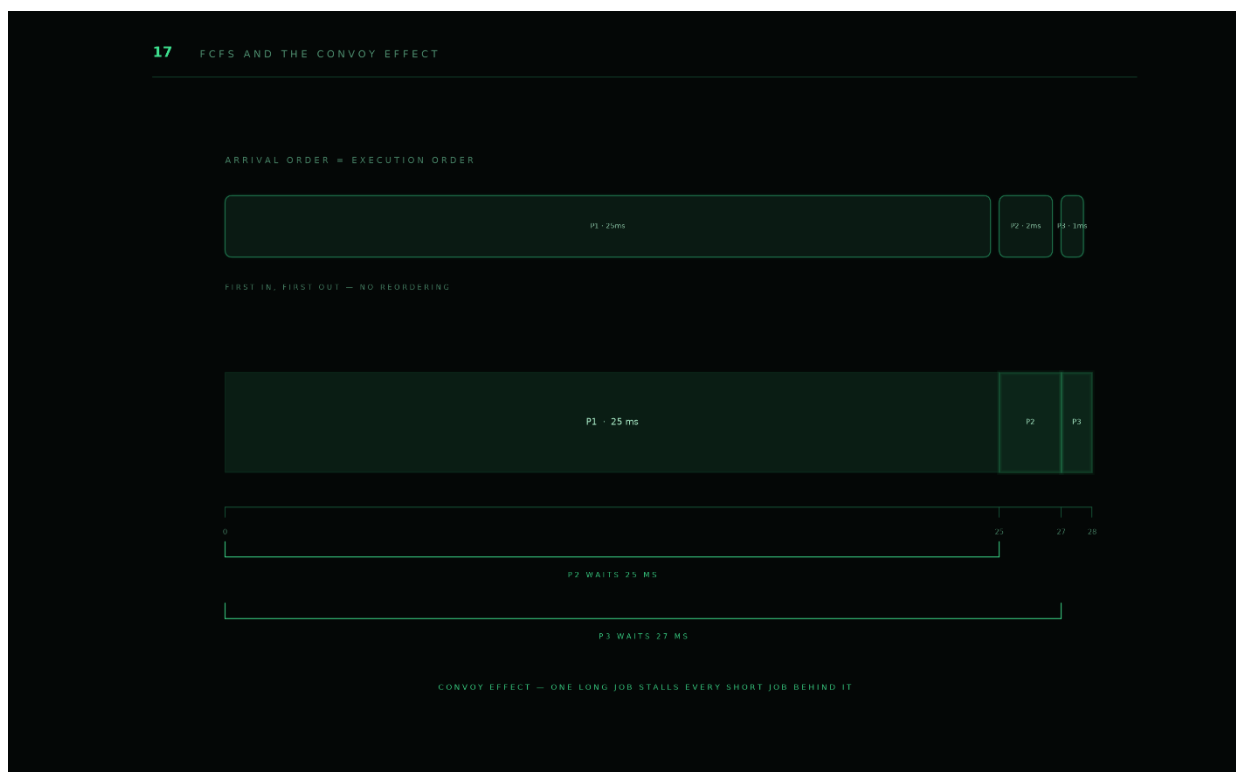
Key Points

- Non-preemptive algorithm.
 - Easy to implement.
 - Not suitable for interactive systems.
-

Interview Follow-up

Q: Which scheduling algorithm is easiest to implement?

A: FCFS.



18. Explain Shortest Job First (SJF) and Shortest Remaining Time First (SRTF).

Answer

Both algorithms prioritize shorter jobs to reduce average waiting time.

Shortest Job First (SJF)

The scheduler selects the process with the smallest CPU burst time.

SJF is **non-preemptive**.

Example

P1 = 8 ms

P2 = 3 ms

P3 = 2 ms

Execution:

P3 → P2 → P1

Advantages

- Minimum average waiting time.
- High throughput.

Disadvantages

- Burst time must be known or estimated.
- Long processes may starve.

Shortest Remaining Time First (SRTF)

SRTF is the **preemptive version of SJF**.

Whenever a new process arrives, the scheduler compares its remaining burst time with the currently running process.

If the new process is shorter, it immediately preempts the current process.

Example

P1 starts (8 ms)

After 2 ms:

P2 arrives (3 ms)

Since $3 < 6$

CPU switches to P2

Comparison

SJF	SRTF
Non-preemptive	Preemptive
Runs until completion	Can be interrupted

SJF

Lower overhead

SRTF

Higher overhead

Key Points

- SJF minimizes average waiting time.
- SRTF improves responsiveness.
- Both may suffer from starvation.

Interview Follow-up

Q: Why isn't SJF commonly used in real systems?

A: Because the operating system usually cannot know the exact future CPU burst time of a process.



19. How does Round Robin scheduling work? How does time quantum affect performance?

Answer

Round Robin (RR) is a **preemptive scheduling algorithm** designed for time-sharing systems.

Each process receives a fixed amount of CPU time called the **time quantum**.

If the process does not finish within that quantum, it is preempted and moved to the end of the ready queue.

Example

Time Quantum = 3 ms

Processes:

P1 = 7 ms

P2 = 5 ms

P3 = 4 ms

Execution:

P1 → P2 → P3 → P1 → P2 → P3 → P1

Effect of Time Quantum

Very Small Quantum

- Excellent responsiveness.

- Too many context switches.
 - High overhead.
-

Very Large Quantum

- Fewer context switches.
 - Behaves similarly to FCFS.
 - Poor responsiveness.
-

Advantages

- Fair scheduling.
 - Suitable for interactive systems.
 - Prevents starvation.
-

Disadvantages

- Performance depends heavily on quantum size.
 - Excessive context switching if quantum is too small.
-

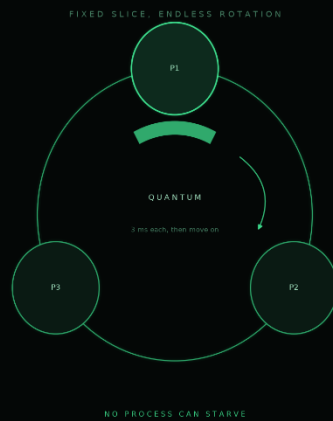
Key Points

- Widely used in time-sharing systems.
 - Every process gets a chance to execute.
 - Choosing an appropriate quantum is important.
-

Interview Follow-up

Q: What happens if the time quantum is extremely large?

A: Round Robin effectively becomes FCFS because processes usually finish before the quantum expires.



$P1 = 7$ $P2 = 5$ $P3 = 4$ $\therefore Q = 3$



$Q \rightarrow 0$



constant switches distribution

$Q \rightarrow \infty$



degenerates into FCFS

20. Explain Priority Scheduling. What is starvation and how can it be prevented?

Answer

In Priority Scheduling, every process is assigned a priority.

The scheduler always executes the highest-priority process first.

Priority may be:

- Static (fixed)
- Dynamic (changes over time)

Priority scheduling can be either preemptive or non-preemptive.

Example

Process Priority

P1 5

P2 2

P3 1

If lower numbers represent higher priority:

Execution:

P3 → P2 → P1

Starvation

Starvation occurs when a low-priority process waits indefinitely because higher-priority processes continue arriving.

Prevention

The most common solution is **Aging**.

The operating system gradually increases the priority of waiting processes.

Eventually, every process receives CPU time.

Advantages

- Important tasks receive immediate attention.
 - Flexible scheduling.
-

Disadvantages

- Low-priority processes may starve.
 - Priority assignment is difficult.
-

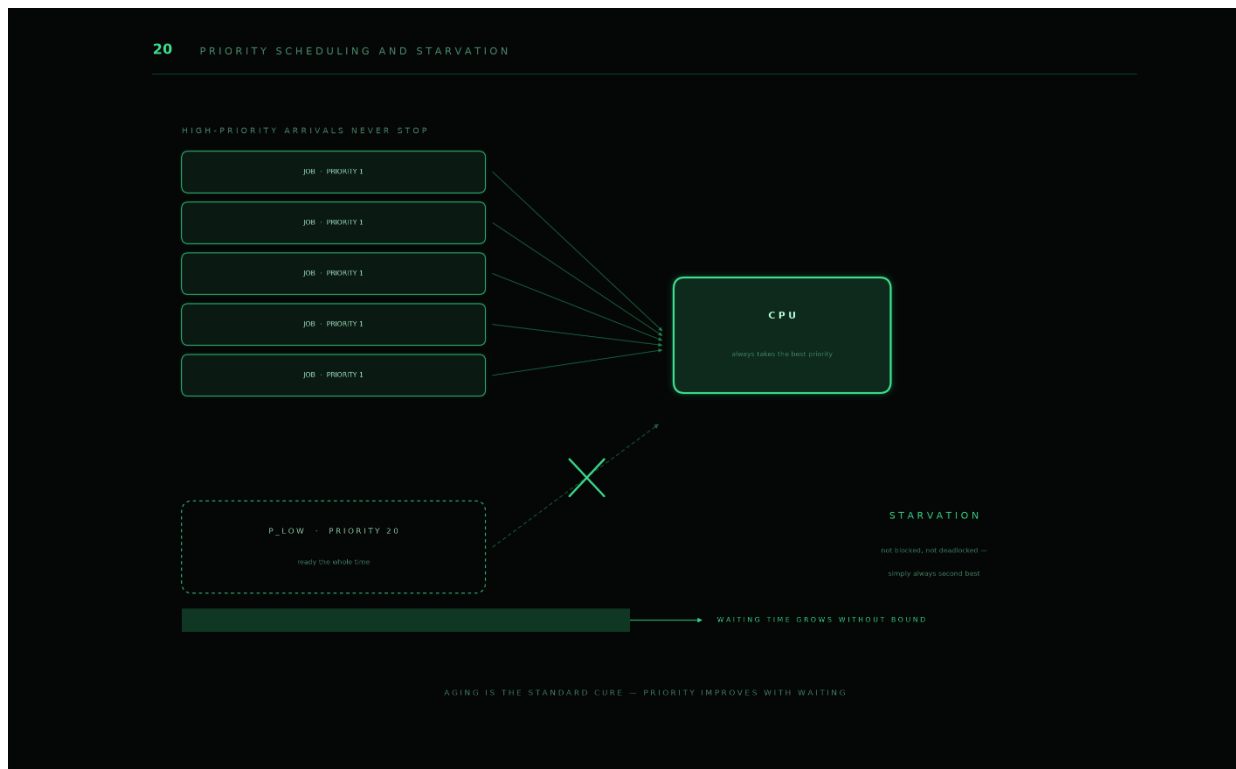
Key Points

- Frequently used in real-time systems.
- Aging prevents indefinite waiting.

Interview Follow-up

Q: Is starvation possible in Round Robin?

A: No. Every process eventually receives CPU time.



21. What is aging in CPU scheduling?

Answer

Aging is a technique used to prevent starvation in priority-based scheduling.

A process waiting in the ready queue gradually receives a higher priority as its waiting time increases. Eventually, its priority becomes high enough to be scheduled.

Example

Suppose:

Priority

$P1 = 1$

P2 = 20

If P2 waits for a long time:

20

↓

18

↓

15

↓

10

↓

5

↓

2

Eventually P2 becomes eligible for execution.

Advantages

- Eliminates starvation.
- Improves fairness.
- Simple to implement.

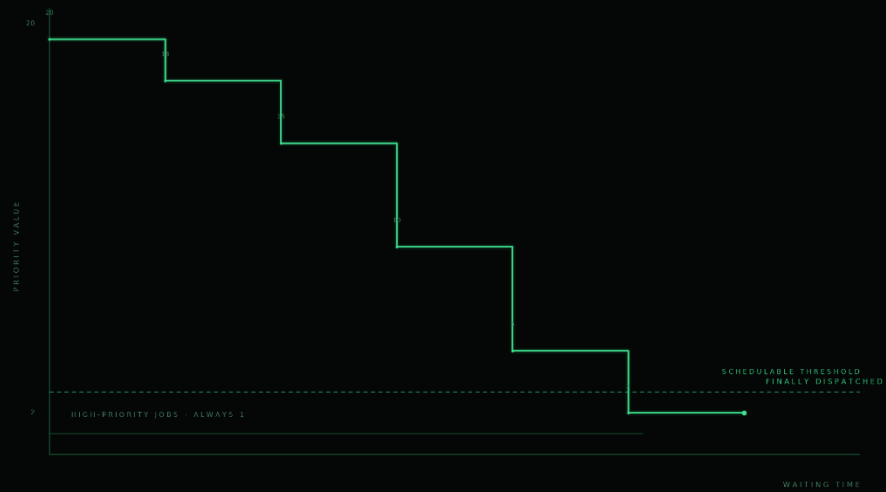
Key Points

- Used primarily with priority scheduling.
- Waiting processes become more important over time.

Interview Follow-up

Q: Does aging increase CPU utilization?

A: Its primary goal is fairness, not CPU utilization.



FAIRNESS, NOT THROUGHPUT – EVERY PROCESS EVENTUALLY WINS

22. What are turnaround time, waiting time, response time, and throughput?

Answer

These metrics evaluate scheduling algorithm performance.

Turnaround Time

The total time taken from process arrival until completion.

Formula

Turnaround Time

=

Completion Time

–

Arrival Time

Waiting Time

The total time a process spends waiting in the ready queue.

Formula

Waiting Time

=

Turnaround Time

–

CPU Burst Time

Response Time

The time from process arrival until it receives the CPU for the first time.

This is especially important for interactive applications.

Throughput

The number of processes completed per unit time.
Higher throughput generally indicates better system performance.

Example

Process:

Arrival = 0

Starts = 4

Completes = 12

Burst = 8

Response Time:

$4 - 0 = 4$

Turnaround Time:

$12 - 0 = 12$

Waiting Time:

$12 - 8 = 4$

Summary Table

Metric	Meaning
Turnaround Time	Arrival → Completion
Waiting Time	Time spent waiting for CPU
Response Time	Arrival → First CPU allocation
Throughput	Completed processes per unit time

Key Points

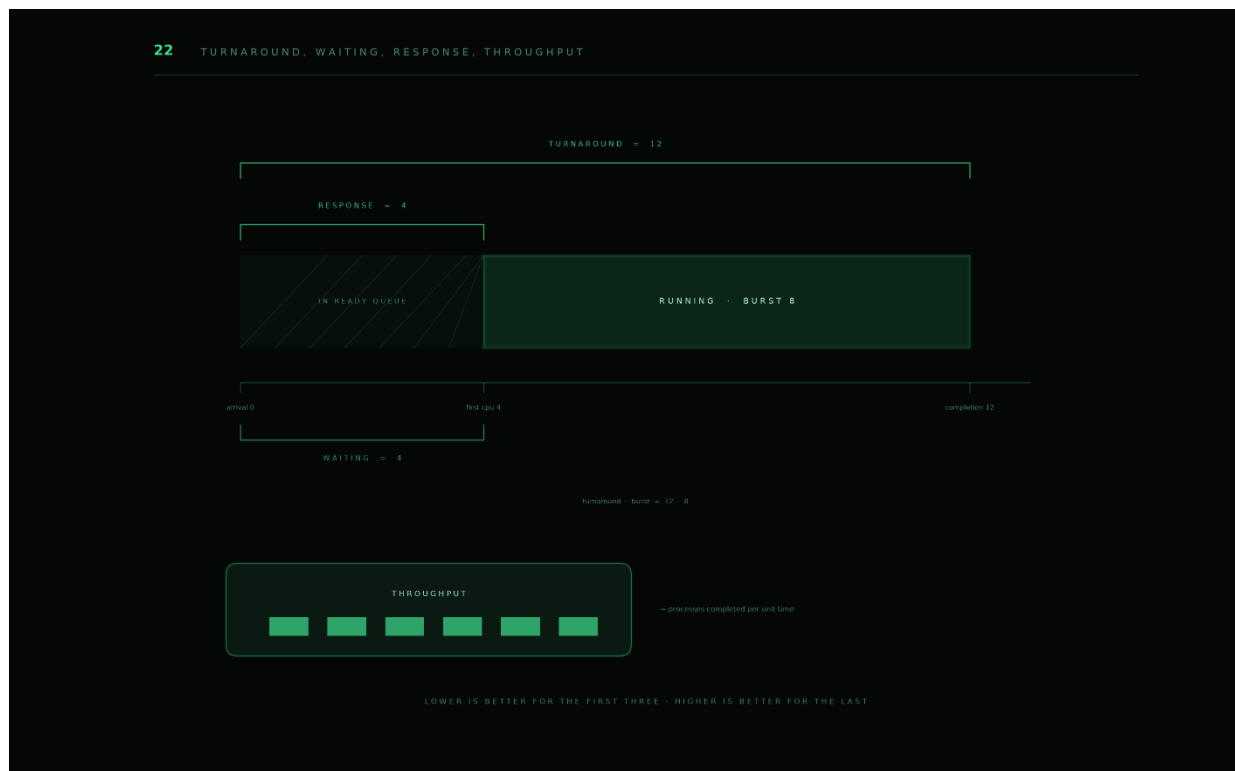
- Lower waiting time is desirable.
- Lower response time improves user experience.
- Higher throughput indicates better resource utilization.

- Different scheduling algorithms optimize different metrics.

Interview Follow-up

Q: Which metric is most important for interactive applications?

A: Response time, because users care about how quickly the application begins responding, even if the total completion time is longer.



Chapter 4 — Synchronization & Concurrency (Q23–Q31)

Modern operating systems allow multiple processes and threads to execute concurrently. While concurrency improves performance and resource utilization, it also introduces challenges when multiple execution units access shared resources. Synchronization mechanisms ensure that shared data remains consistent and free from unexpected behavior.

23. What is a race condition? Give a practical example.

Answer

A **race condition** occurs when two or more processes or threads access and modify the same shared resource simultaneously, and the final result depends on the order in which they execute.

Since thread scheduling is unpredictable, the output may vary from one execution to another.

Race conditions often lead to incorrect results, inconsistent data, or application crashes.

Example

Suppose two threads update the same bank account balance.

Initial Balance:

Balance = 1000

Thread A:

Read Balance → 1000

Add 500

Write 1500

Thread B:

Read Balance → 1000

Subtract 200

Write 800

If both threads read the balance before either writes it back, the final balance could incorrectly become **800** or **1500** instead of the expected **1300**.

Why It Happens

A typical update operation consists of three steps:

1. Read value
2. Modify value
3. Write value back

If another thread interrupts between these steps, inconsistent results can occur.

How to Prevent It

- Mutexes
 - Semaphores
 - Monitors
 - Atomic operations
 - Locks
-

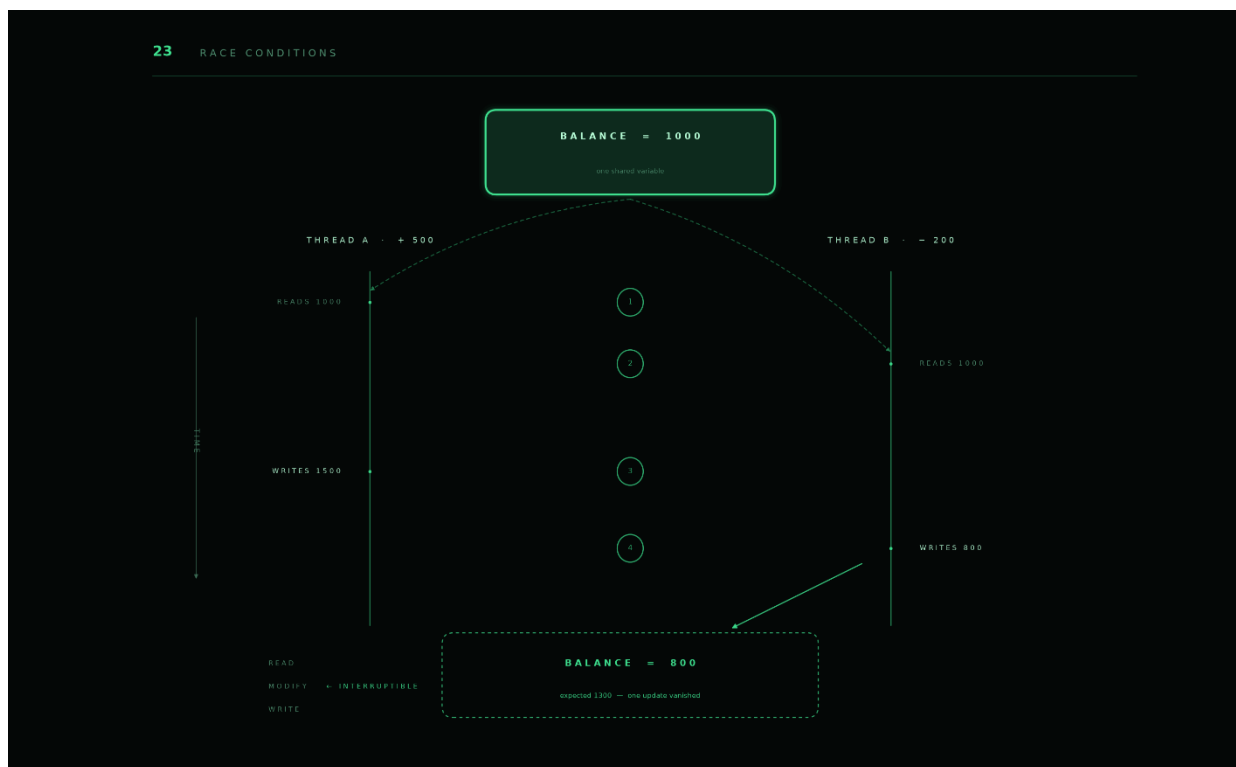
Key Points

- Occurs only when shared data is accessed concurrently.
 - Execution order affects the final outcome.
 - Proper synchronization prevents race conditions.
-

Interview Follow-up

Q: Does every multithreaded program have race conditions?

A: No. Race conditions occur only when multiple threads access shared data without proper synchronization.



24. What is the critical section problem?

Answer

A **critical section** is the part of a program where shared resources such as variables, files, or data structures are accessed.

The **critical section problem** is the challenge of ensuring that only one process or thread executes its critical section at a time.

Without proper synchronization, concurrent access may cause race conditions.

Example

Suppose multiple threads update:

Total Sales

If two threads modify this value simultaneously, incorrect totals may be produced.

The update operation should therefore be protected inside a critical section.

Structure of a Process

Entry Section



Critical Section



Exit Section



Remaining Code

- **Entry Section:** Requests permission to enter.
- **Critical Section:** Accesses shared resource.
- **Exit Section:** Releases the resource.
- **Remaining Section:** Executes normal code.

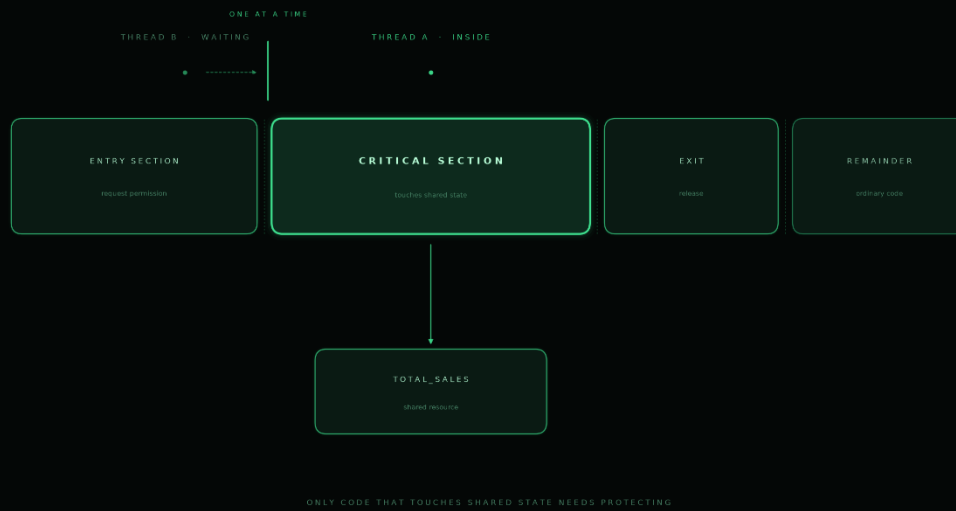
Key Points

- Only one thread should execute a critical section at a time.
- Synchronization mechanisms protect critical sections.
- Critical sections help maintain data consistency.

Interview Follow-up

Q: Is every piece of code a critical section?

A: No. Only code that accesses shared resources requires protection.



25. What conditions must a correct critical section solution satisfy?

Answer

A correct synchronization solution must satisfy **three essential conditions**.

1. Mutual Exclusion

Only one process or thread can execute its critical section at any given time.

If Thread A is updating shared data, Thread B must wait.

2. Progress

If no process is inside the critical section, the decision about who enters next should not be postponed indefinitely.

Processes that are not interested in entering the critical section should not influence the decision.

3. Bounded Waiting

Every process requesting access should eventually enter the critical section.

No process should wait forever.

Why These Conditions Matter

Without:

- Mutual exclusion → race conditions occur.
 - Progress → unnecessary delays occur.
 - Bounded waiting → starvation may occur.
-

Key Points

A good synchronization algorithm guarantees:

- Safety
 - Fairness
 - Progress
-

Interview Follow-up

Q: Which condition prevents starvation?

A: Bounded Waiting.



MUTUAL EXCLUSION GIVES SAFETY · PROGRESS GIVES LIVENESS · BOUNDED WAITING GIVES FAIRNESS

DROP BOUNDED WAITING AND STARVATION RETURNS

26. What is a semaphore? Differentiate between binary and counting semaphores.

Answer

A **semaphore** is a synchronization mechanism used to control access to shared resources.

It maintains an integer value that is modified using two atomic operations:

- **Wait (P)** → Decreases the semaphore value.
- **Signal (V)** → Increases the semaphore value.

If the semaphore value is unavailable, the requesting thread waits.

Binary Semaphore

A binary semaphore can have only two values:

0 or 1

It behaves similarly to a mutex and is typically used for protecting a single resource.

Counting Semaphore

A counting semaphore can hold values greater than one.

It allows multiple processes to access a limited number of identical resources.

Example

Suppose there are:

5 printers

Semaphore value:

5

Each print request decreases the count.

When printing finishes, the semaphore value increases.

Comparison

Binary Semaphore	Counting Semaphore
------------------	--------------------

Values: 0 or 1	Values: 0 to N
----------------	----------------

One resource	Multiple resources
--------------	--------------------

Similar to mutex	Resource counting
------------------	-------------------

Binary Semaphore Counting Semaphore

Simpler

More flexible

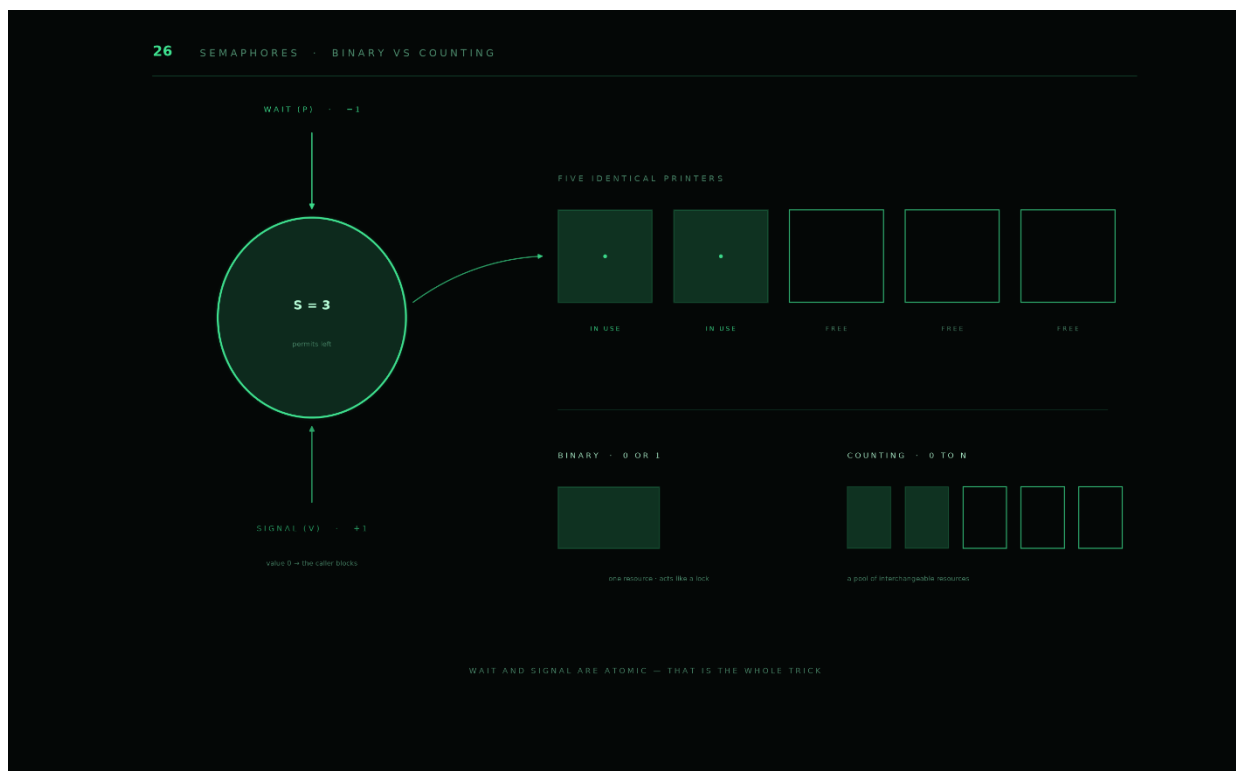
Key Points

- Semaphores synchronize concurrent execution.
- Binary semaphores protect single resources.
- Counting semaphores manage multiple identical resources.

Interview Follow-up

Q: Can a semaphore have a negative value?

A: Internally, some implementations may represent waiting processes with negative values, but conceptually a semaphore represents available resource count.



27. What is a mutex? How is it different from a semaphore?

Answer

A **mutex (Mutual Exclusion Lock)** is a synchronization object that allows only one thread to access a shared resource at a time.

Unlike semaphores, a mutex has ownership.

Only the thread that locks a mutex is allowed to unlock it.

Working

Thread A:

Lock Mutex



Critical Section



Unlock Mutex

While Thread A holds the mutex, all other threads attempting to acquire it must wait.

Mutex vs Semaphore

Mutex	Semaphore
Ownership exists	No ownership
Only owner unlocks	Any thread may signal
One resource	One or multiple resources
Simpler	More flexible

Example

Updating a shared linked list:

Only one thread should modify the list at a time.

A mutex is ideal for this scenario.

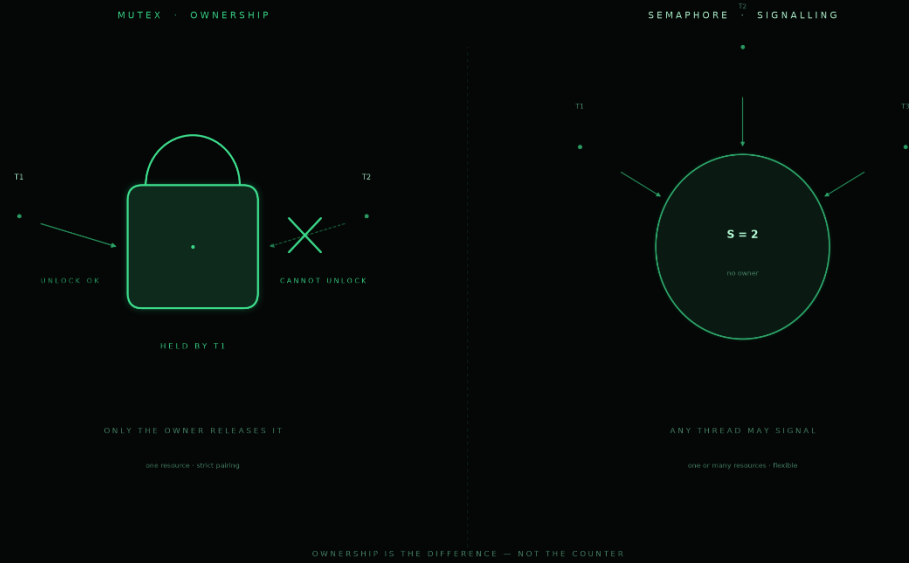
Key Points

- Mutex protects critical sections.
 - Mutex ensures exclusive access.
 - Ownership distinguishes mutexes from semaphores.
-

Interview Follow-up

Q: When should you choose a mutex instead of a semaphore?

A: When only one thread should own and protect a shared resource.



28. Explain monitors and condition variables.

Answer

A **monitor** is a high-level synchronization construct that combines:

- Shared data
- Synchronization methods
- Automatic mutual exclusion

Only one thread can execute inside a monitor at a time.

Monitors simplify synchronization by hiding low-level locking details.

Condition Variables

A condition variable allows threads to wait until a specific condition becomes true.

Two common operations are:

- **wait()** → Releases the monitor lock and waits.
- **signal()** → Wakes one waiting thread.

Some implementations also provide **broadcast()**, which wakes all waiting threads.

Example

Suppose a bounded queue is empty.

A consumer thread calls:

`wait()`

When a producer inserts an item:

`signal()`

The waiting consumer wakes up and continues execution.

Advantages

- Easier to use than semaphores.
 - Less error-prone.
 - Built into many programming languages.
-

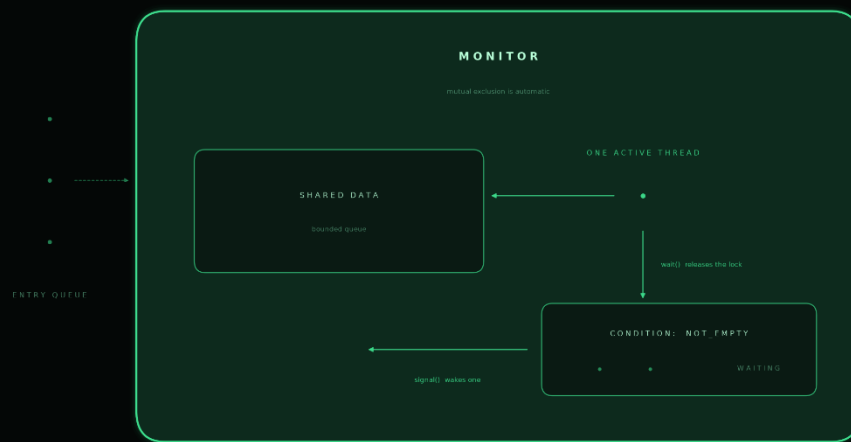
Key Points

- Monitors provide automatic mutual exclusion.
 - Condition variables coordinate waiting and notification.
 - Widely used in Java and modern synchronization libraries.
-

Interview Follow-up

Q: Can multiple threads execute inside the same monitor simultaneously?

A: No. A monitor allows only one active thread at a time.



A LOCK YOU CANNOT FORGET TO TAKE — PLUS A PLACE TO WAIT

BROADCAST WAKES EVERY WAITER INSTEAD OF ONE

29. Explain the Producer-Consumer problem and one possible solution.

Answer

The **Producer-Consumer Problem** describes two types of processes sharing a finite buffer.

- **Producer** generates data.
- **Consumer** processes the data.

The challenge is to ensure:

- Producers do not insert into a full buffer.
- Consumers do not remove from an empty buffer.
- Both access the buffer safely.

Example

Producer

↓

Shared Buffer

↓

Consumer

Typical Solution

Use:

- One mutex
- Two counting semaphores

Semaphores:

Empty = Buffer Size

Full = 0

Producer:

1. Wait for empty slot.

2. Lock mutex.
3. Insert item.
4. Unlock mutex.
5. Signal full.

Consumer:

1. Wait for full slot.
2. Lock mutex.
3. Remove item.
4. Unlock mutex.
5. Signal empty.

Applications

- Message queues
- Print spoolers
- Streaming systems
- Logging systems

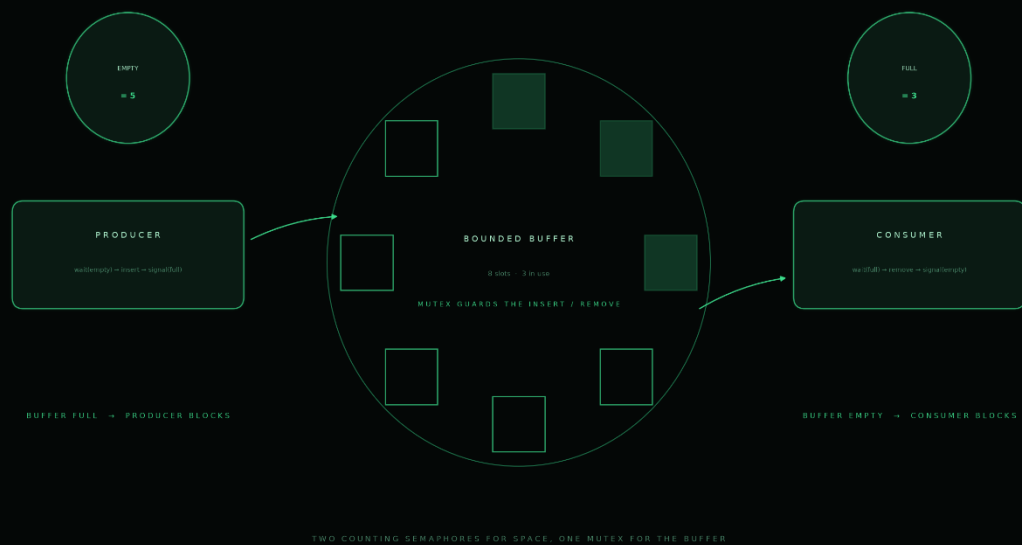
Key Points

- Prevents buffer overflow.
- Prevents buffer underflow.
- Demonstrates synchronization between cooperating threads.

Interview Follow-up

Q: Which synchronization primitives are commonly used?

A: Mutexes combined with semaphores or condition variables.



30. Explain the Readers-Writers problem.

Answer

The **Readers-Writers Problem** involves multiple processes accessing shared data.

There are two types of operations:

- **Readers** only read the data.

- **Writers** modify the data.

Multiple readers can safely access the data simultaneously because reading does not change the data.

However, writing requires exclusive access.

Rules

- Many readers can read together.
 - Only one writer may write.
 - No reader can read while a writer is writing.
-

Example

Database:

Readers

↓

Database

↑

Writer

Several users can view product information at the same time.

If an administrator updates prices, all readers must wait until the update finishes.

Common Variants

Reader Preference

Readers never wait unless a writer is already writing.

Problem:

Writers may starve.

Writer Preference

Waiting writers are given priority.

Problem:

Readers may experience delays.

Fair Solution

Both readers and writers eventually receive access.

Applications

- Databases
 - Search engines
 - Configuration systems
 - Shared caches
-

Key Points

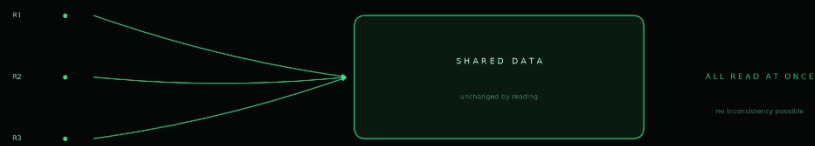
- Multiple readers are allowed simultaneously.
 - Writers require exclusive access.
 - Synchronization policies affect fairness and performance.
-

Interview Follow-up

Q: Why are multiple readers allowed?

A: Reading does not modify shared data, so concurrent reads do not create inconsistencies.

READERS · CONCURRENT



WRITER · EXCLUSIVE



READER PREFERENCE STARVES WRITERS · WRITER PREFERENCE DELAYS READERS

31. Explain the Dining Philosophers problem and why it is important.

Answer

The **Dining Philosophers Problem** is a classic synchronization problem used to demonstrate deadlocks and resource allocation issues.

Imagine five philosophers sitting around a circular table.

Between every pair of philosophers lies one fork.

Each philosopher alternates between:

- Thinking
- Eating

To eat, a philosopher needs **both adjacent forks**.

Problem

If every philosopher picks up the left fork simultaneously, each waits forever for the right fork.

This creates a deadlock.

Illustration

P1 Fork P2

Fork Fork

P5 P3

Fork P4

Possible Solutions

Resource Ordering

Always pick up the lower-numbered fork first.

Limit Concurrency

Allow at most four philosophers to attempt eating simultaneously.

Waiter Solution

A central coordinator grants permission before forks are picked up.

Asymmetric Solution

Odd philosophers pick up the left fork first.

Even philosophers pick up the right fork first.

Why Is It Important?

The problem illustrates:

- Deadlock
- Starvation
- Resource allocation
- Synchronization
- Fair scheduling

Many real-world systems, such as databases and operating systems, use similar concepts when managing locks.

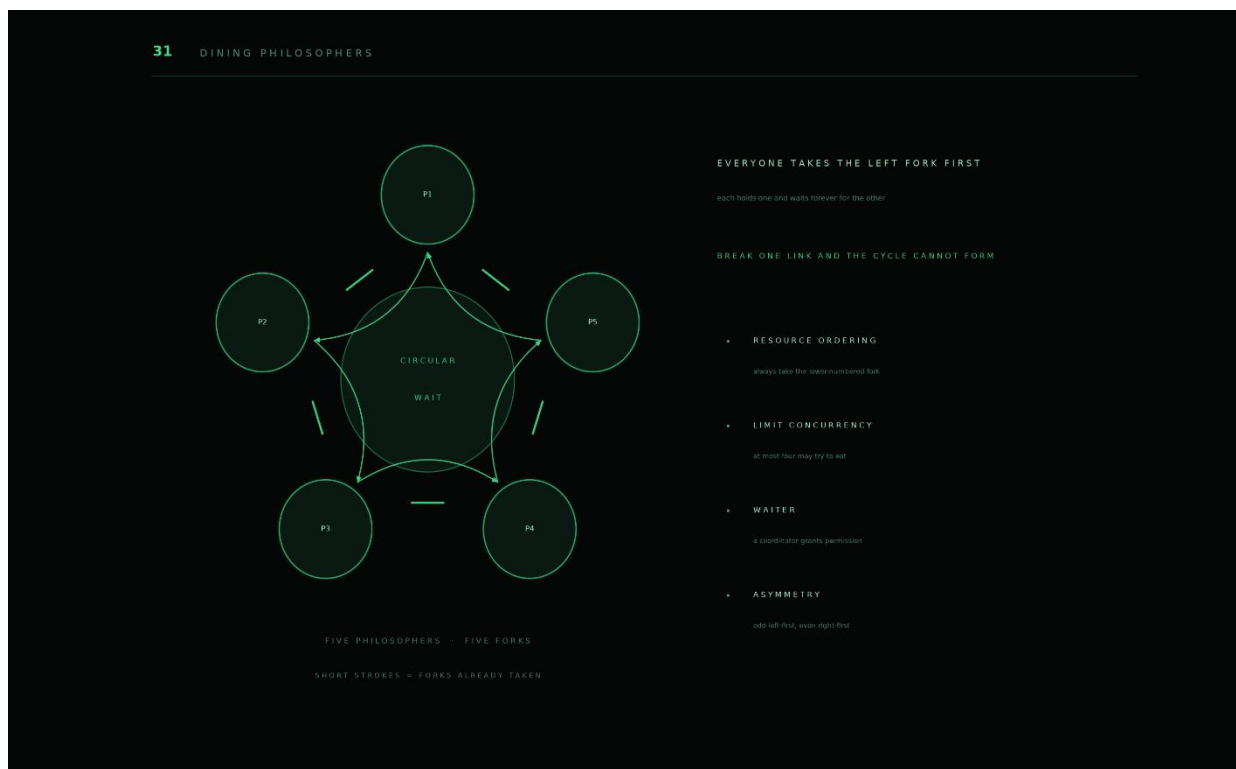
Key Points

- Demonstrates deadlock scenarios.
- Highlights the importance of careful resource allocation.
- Frequently used in operating systems and distributed systems education.

Interview Follow-up

Q: Does the Dining Philosophers problem always lead to deadlock?

A: No. Deadlock occurs only under certain resource acquisition strategies. Proper synchronization techniques can prevent it entirely.



Chapter 4 Summary

This chapter introduced the core synchronization concepts used in concurrent systems:

- **Race Conditions** occur when multiple threads access shared data without coordination.

- **Critical Sections** identify code that must execute exclusively.
- **Mutual Exclusion, Progress, and Bounded Waiting** define the requirements of a correct synchronization solution.
- **Semaphores** and **Mutexes** are fundamental synchronization primitives, while **Monitors** provide a higher-level abstraction.
- Classic problems such as **Producer-Consumer**, **Readers-Writers**, and **Dining Philosophers** illustrate real-world synchronization challenges and their solutions.

Chapter 5 — Deadlocks (Q32–Q36)

A deadlock is one of the most important concepts in Operating Systems. It occurs when a group of processes becomes permanently blocked because each process is waiting for a resource held by another process. Since none of the processes can proceed, the system reaches a standstill.

Understanding deadlocks helps in designing systems that remain reliable even when multiple processes compete for limited resources.

32. What is a deadlock?

Answer

A **deadlock** is a situation in which two or more processes are unable to continue execution because each is waiting for a resource currently held by another process in the same group.

Since no process can obtain the required resource, all of them remain blocked indefinitely.

Deadlocks commonly occur when multiple processes compete for limited resources such as files, printers, memory, or locks.

Example

Suppose two processes require two resources:

Process A → Printer → Scanner

Process B → Scanner → Printer

Execution:

1. Process A acquires the printer.
2. Process B acquires the scanner.
3. Process A waits for the scanner.
4. Process B waits for the printer.

Neither process can continue, resulting in a deadlock.

Real-World Analogy

Imagine two people trying to cross a narrow bridge from opposite directions.

Neither is willing to move back.

Both remain stuck until someone gives way.

Effects of Deadlock

- Processes stop executing.
- System resources remain occupied.
- Throughput decreases.
- Some applications may become unresponsive.

Key Points

- Processes wait indefinitely.
- Resources remain allocated but unused.
- External intervention may be required to recover.

Interview Follow-up

Q: Does every waiting process indicate a deadlock?

A: No. Waiting is normal. A deadlock occurs only when a circular dependency prevents all involved processes from making progress.



33. What are the four necessary conditions for deadlock?

Answer

A deadlock can occur only if **all four** of the following conditions hold simultaneously.

If even one condition is removed, deadlock cannot occur.

1. Mutual Exclusion

At least one resource must be non-shareable.

Only one process can use the resource at a time.

Example

A printer can print only one document at a time.

2. Hold and Wait

A process holds one or more resources while waiting to acquire additional resources.

Example

A process holds a file lock while waiting for a printer.

3. No Preemption

Resources cannot be forcibly taken away.

They must be released voluntarily by the process holding them.

Example

The operating system cannot abruptly take away a file lock that is actively being used.

4. Circular Wait

A circular chain of waiting processes exists.

Example:

P1 waits for P2

↓

P2 waits for P3

↓

P3 waits for P1

Each process waits for a resource held by the next process.

Summary

Condition	Meaning
Mutual Exclusion	Resource cannot be shared
Hold and Wait	Holding one resource while requesting another
No Preemption	Resources cannot be forcibly removed
Circular Wait	Processes form a waiting cycle

Key Points

- All four conditions are necessary.

- Eliminating any one condition prevents deadlock.
- Most prevention techniques target one of these conditions.

Interview Follow-up

Q: Which condition is easiest to eliminate?

A: It depends on the system, but breaking the circular wait condition by enforcing a fixed resource acquisition order is a common and practical approach.



34. Compare deadlock prevention, avoidance, detection, and recovery.

Answer

Operating systems use four major strategies to deal with deadlocks.

Each strategy has different trade-offs between safety, performance, and implementation complexity.

1. Deadlock Prevention

The operating system ensures that at least one of the four necessary conditions can never occur.

Example

Require processes to request all required resources at once before execution.

Advantages

- Deadlocks never occur.

Disadvantages

- Poor resource utilization.
 - Processes may hold resources they do not immediately need.
-

2. Deadlock Avoidance

Resources are allocated only if doing so keeps the system in a **safe state**.

The operating system predicts whether granting a request could eventually lead to a deadlock.

The most famous avoidance algorithm is the **Banker's Algorithm**.

Advantages

- Better resource utilization than prevention.

- Avoids unsafe allocations.

Disadvantages

- Requires advance knowledge of maximum resource needs.
 - Computationally expensive.
-

3. Deadlock Detection

The operating system allows deadlocks to occur and periodically checks whether one exists.

If a deadlock is detected, corrective action is taken.

Advantages

- Simple resource allocation.
- Higher resource utilization.

Disadvantages

- Detection itself consumes system resources.
 - Recovery can be expensive.
-

4. Deadlock Recovery

After detecting a deadlock, the operating system attempts to remove it.

Possible recovery methods include:

- Terminating one or more processes.
 - Releasing resources.
 - Rolling back affected processes.
-

Comparison Table

Strategy	Deadlocks Allowed?	Main Idea
Prevention	No	Remove one necessary condition
Avoidance	No	Allocate resources safely
Detection	Yes	Detect deadlocks after they occur
Recovery	Yes	Eliminate detected deadlocks

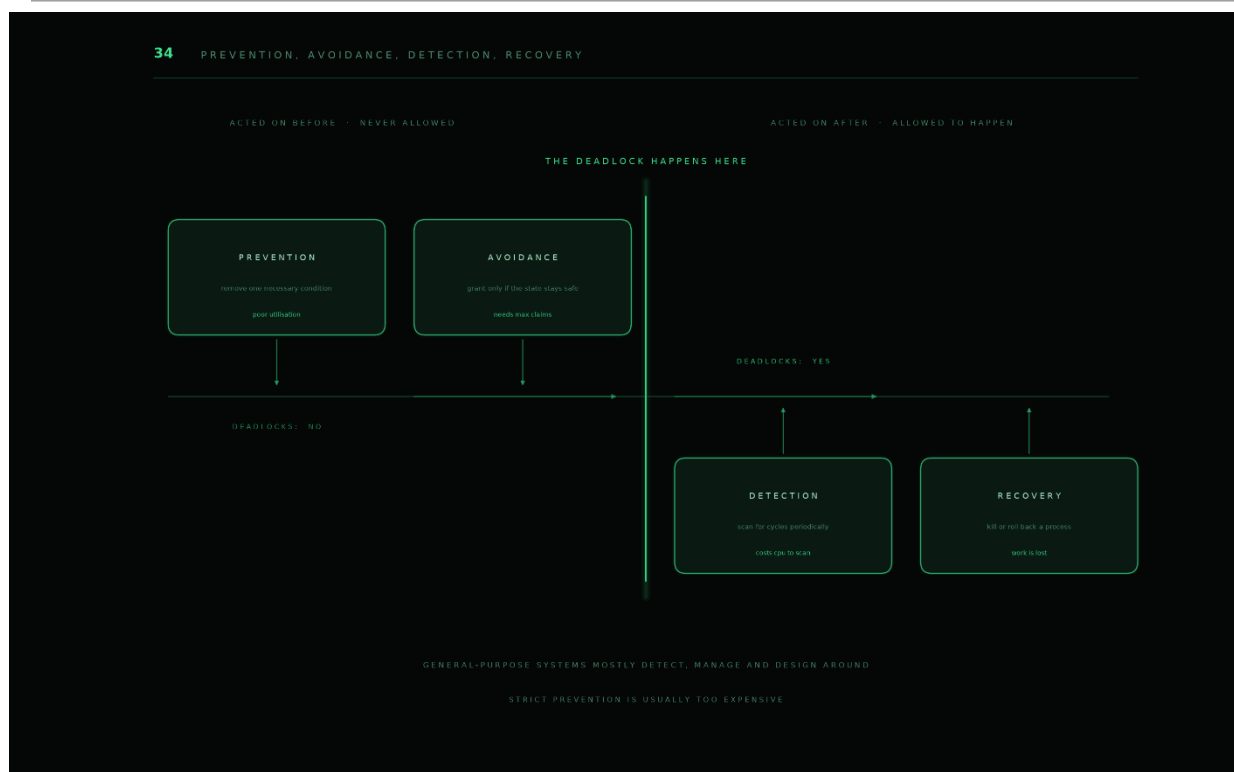
Key Points

- Prevention is proactive.
- Avoidance is predictive.
- Detection is reactive.
- Recovery resolves detected deadlocks.

Interview Follow-up

Q: Which strategy is most commonly used in modern operating systems?

A: General-purpose operating systems often rely on a combination of detection, careful resource management, and application-level design rather than implementing strict deadlock prevention or avoidance for all resources.



35. What is the Banker's Algorithm? Where is it applicable?

Answer

The **Banker's Algorithm** is a deadlock avoidance algorithm developed by Edsger Dijkstra.

It determines whether granting a resource request will leave the system in a **safe state**.

If the allocation keeps the system safe, the request is granted.

Otherwise, the requesting process must wait.

Why the Name "Banker"?

Imagine a bank that lends money.

The bank approves a loan only if it can still satisfy the future needs of all customers without running out of funds.

Similarly, the operating system grants resources only if it can still satisfy all processes safely.

Information Required

For every process, the operating system maintains:

- Maximum resources required
 - Resources currently allocated
 - Remaining resource needs
 - Available resources
-

Safe State

A system is in a **safe state** if there exists an order in which every process can complete successfully without causing deadlock.

Unsafe State

An unsafe state does **not necessarily** mean a deadlock has occurred.

It simply means the system can no longer guarantee that deadlock can be avoided.

Example

Available memory:

10 Units

Process A:

Maximum:

7

Allocated:

3

Remaining Need:

4

The algorithm checks whether allocating more memory keeps the system safe.

Advantages

- Prevents deadlocks.
 - Maximizes safe resource utilization.
-

Limitations

- Requires advance knowledge of maximum resource requirements.
 - Expensive for large systems.
 - Not practical for many modern applications where resource needs change dynamically.
-

Key Points

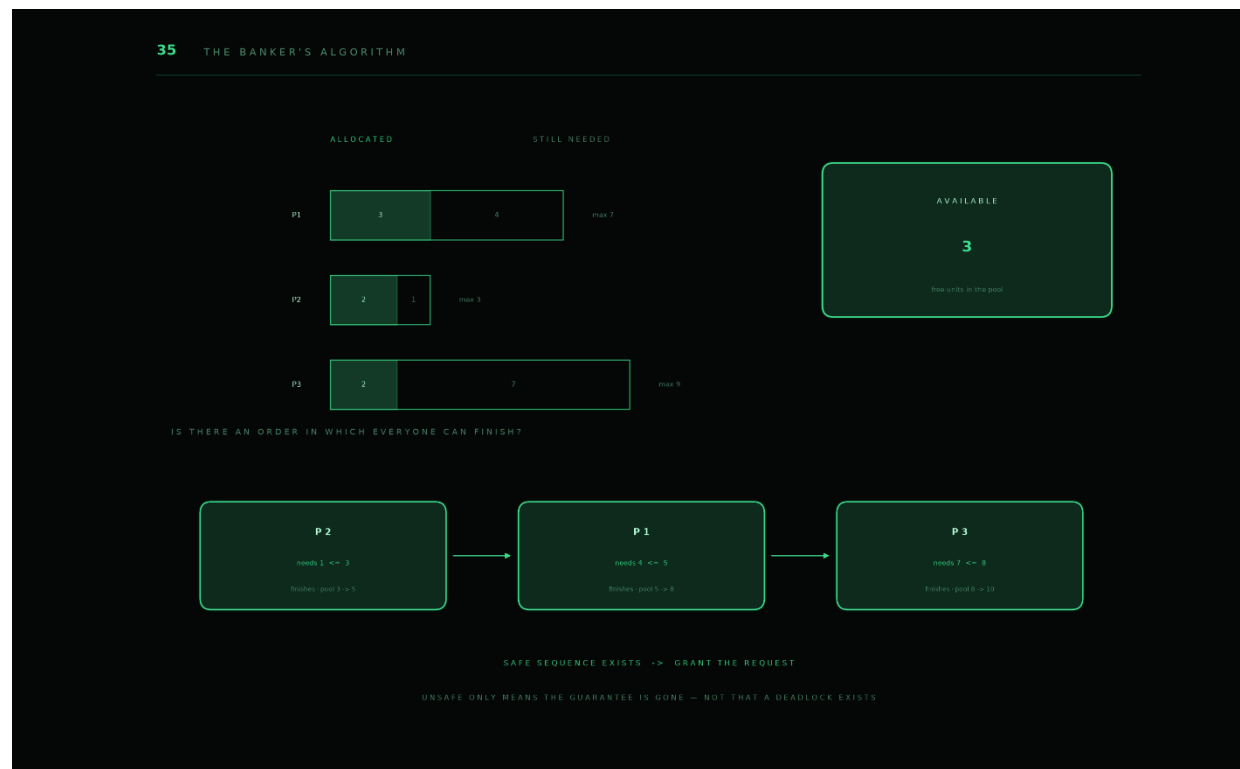
- Deadlock avoidance algorithm.

- Uses the concept of safe states.
- Requires maximum resource claims in advance.

Interview Follow-up

Q: Does an unsafe state always imply a deadlock?

A: No. An unsafe state indicates the possibility of future deadlock, not that one already exists.



36. What is starvation? How is it different from deadlock?

Answer

Although starvation and deadlock both prevent progress, they occur for different reasons.

Starvation

Starvation occurs when a process waits indefinitely because other processes continue receiving priority or resources.

The process is continuously postponed even though the system itself continues to function.

Example

Suppose high-priority tasks keep arriving.

A low-priority process may never receive CPU time.

The scheduler continues running other processes, causing the low-priority process to starve.

Deadlock

In a deadlock, multiple processes wait for one another in a circular dependency.

None of the involved processes can proceed.

Comparison

Starvation

One or more processes wait indefinitely

System continues running

Caused by unfair scheduling

Deadlock

Group of processes wait indefinitely

Processes involved stop permanently

Caused by circular resource dependency

Starvation

Aging can prevent it

Deadlock

Requires deadlock handling strategies

Example

Starvation

High Priority Jobs



High Priority Jobs



High Priority Jobs



Low Priority Job (Never Executes)

Deadlock

P1 waits for P2



P2 waits for P1

Both remain blocked forever.

Preventing Starvation

The most common solution is **Aging**, where waiting processes gradually receive higher priority.

Eventually, every process gets a chance to execute.

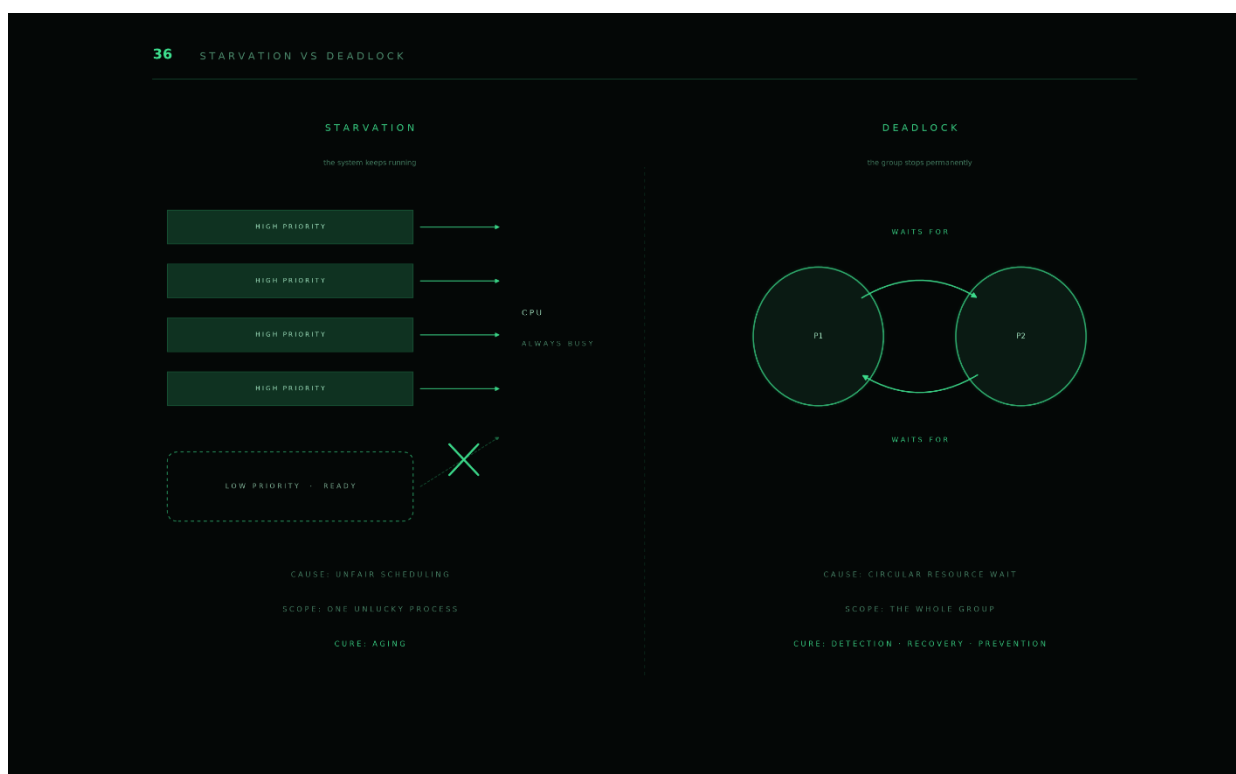
Key Points

- Starvation is caused by unfair scheduling.
- Deadlock is caused by circular waiting.
- Aging prevents starvation but does not resolve deadlocks.

Interview Follow-up

Q: Can starvation occur without a deadlock?

A: Yes. A process may starve because of scheduling policies even though no circular dependency exists.



Chapter 5 Summary

Deadlocks arise when multiple processes compete for limited resources in a way that prevents any of them from making progress. Understanding the **four necessary conditions**—Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait—is essential because removing any one of them prevents deadlock.

Operating systems use four main strategies to handle deadlocks:

- **Prevention** eliminates one of the necessary conditions.
- **Avoidance** allocates resources only when the system remains in a safe state, using algorithms such as the **Banker's Algorithm**.
- **Detection** allows deadlocks to occur and identifies them later.
- **Recovery** resolves detected deadlocks by reclaiming resources or terminating processes.

Finally, **starvation** differs from deadlock because it results from unfair scheduling rather than circular

resource dependencies. Techniques like **aging** improve fairness and help ensure that every process eventually receives CPU time.

This chapter provides the conceptual foundation needed to understand how operating systems safely manage shared resources in concurrent environments.